

Applied Machine Learning (CSAI2017P)

Mid-Semester Study Material

The examinable topics of Lectures 1–9, worked by hand — with the course’s practice questions, exercises and answers

Dr. Tofik Ali

School of Computer Science, UPES Dehradun

Autumn 2026 • Lecture packages L1–L9 (Units I–IV and Unit V to Lecture 9)

Read this first

Syllabus. The mid-semester paper covers Lectures 1–9: Unit I (L1), Unit II (L2), Unit III (L3–L4), Unit IV (L5–L7) and the first two lectures of Unit V (L8, regression foundations and least squares; L9, maximum likelihood, R^2 , model adequacy and over-fitting). **Not examined** at mid-semester: the classification and margin losses of Lecture 2 — binary, categorical and sparse categorical cross-entropy, hinge (SVM) loss and triplet loss — and everything from Lecture 10 onwards (logistic regression, multiple regression, regularisation). Adjusted R^2 with a given number of features p is examined; fitting a multiple regression is not.

What is inside. Twenty-one topics, in lecture order. Each has **the method** as a numbered recipe and **worked examples** done in full; most also list the **mistakes** that cost marks and end with **exercises** — numerical ones in a purple box (answers at the back) and short-answer ones with model answers in a grey box. The exercises draw on the course’s shared practice material (the chapter notes, the *Practice & Worked Solutions* workbook and Class Test Set B), recreated here with every answer checked. Two complete practice papers with solutions close the booklet.

How to use it. Attempt every exercise on paper before you look at Section 23. Reading a solution feels like understanding; only doing it is.

Topic	Lecture	Topic	Lecture
1 What ML is; types of learning	L1	10 Outliers and transformations	L5
2 Regression losses	L2	11 Feature scaling	L6
3 Gradient descent by hand	L3	12 Encoding and feature engineering	L6
4 Batch, stochastic, mini-batch	L3	13 Data reduction and PCA	L6
5 Momentum	L3	14 Splitting and cross-validation	L7
6 Adagrad and RMSprop	L4	15 Imbalanced data	L7
7 Adam	L4	16 Data leakage and pipeline order	L5–L7
8 Adadelat; comparing optimisers	L4	17 Regression foundations	L8
9 Missing data	L5	18 Least squares by hand	L8
		19 Direct regression; max. likelihood	L9
		20 R^2 , adjusted R^2 , error summary	L9
		21 Adequacy, over-fitting, CV	L9

How numericals are marked

A correct final number with no working earns little; a correct method with a slip in one line earns most of the marks. Write the method visibly — the residual column, the gradient at each step, the accumulator, the sorted data and the quartiles, the centred table — carry **four decimal places**, and round only at the end. Finish every answer with a **check**: residuals that sum to zero, a first Adam step of exactly η , a line that passes through $(\bar{x}, \bar{y})$.

Conventions used throughout

- **Residual** $e = y - \hat{y}$ (actual minus predicted). Positive means the model *under*-predicted.
- **Standard deviation.** Standardisation (**StandardScaler**) uses the population σ , divisor n . The z -score outlier rule and the sample sd of a column use s , divisor $n - 1$.
- **Quartiles by hand:** sort, split into a lower and an upper half (for odd n leave the median out of both), and take the median of each half (Tukey's hinges). NumPy's default `np.percentile` interpolates and can differ slightly; either is accepted if you say which you used.
- **Optimiser toy problem.** Unless stated otherwise the optimiser sections use two targets $y = 1$ and $y = 3$, a constant model w , half-squared row loss $\frac{1}{2}(w - y)^2$ and their mean $J(w) = \frac{1}{2}(w - 2)^2 + \frac{1}{2}$, so $J'(w) = w - 2$. With a loss $(w - c)^2$ (no half) the gradient is $2(w - c)$ — say which you use.
- **Momentum** is $v \leftarrow \beta v + g$, $w \leftarrow w - \eta v$. Adam's first moment is $m \leftarrow \beta_1 m + (1 - \beta_1)g$. Do not mix them.

Contents

1	What machine learning is, and the three types of learning	4
2	Regression losses: SSE, MSE, RMSE, MAE and Huber	6
3	Gradient descent by hand	8
4	Batch, stochastic and mini-batch gradient descent	9
5	Momentum	10
6	Adagrad and RMSprop	11
7	Adam, with bias correction	13
8	Adadelta, and comparing optimisers	14
9	Missing data	15
10	Outliers and transformations	17
11	Feature scaling	18
12	Encoding and feature engineering	20
13	Data reduction and PCA	21
14	Splitting and cross-validation	22

15 Imbalanced data	24
16 Data leakage and the order of the pipeline	25
17 Regression foundations	26
18 Least squares by hand	27
19 Direct regression and maximum likelihood	29
20 R^2 , adjusted R^2 and the error summary	31
21 Model adequacy, over-fitting and cross-validation	32
22 Practice papers	33
23 Answers to the numerical exercises	35

Unit I — Introduction to machine learning (L1)

1 What machine learning is, and the three types of learning

The method: naming and classifying a learning problem

1. **Definition** (Mitchell): a program learns from experience E on task T , measured by P , if its performance at T , as measured by P , improves with E . Name all three, and make P specific.
2. **Decide the type from the feedback.** Known answers in the training table $\Rightarrow$ *supervised*; no answers, find structure $\Rightarrow$ *unsupervised*; rewards for actions $\Rightarrow$ *reinforcement*.
3. **If supervised, look at the target column:** a number on a continuous scale $\Rightarrow$ regression; a category $\Rightarrow$ classification. The target decides, not the features.
4. **Say what one training row contains:** the features, and the target if there is one.
5. **Vocabulary:** a *sample* is a row, a *feature* a column used as input, the *target* the column to predict. A *parameter* is learned from data (a slope b_1); a *hyperparameter* is set before learning (the k of k -NN, a learning rate).
6. **Judge generalisation on data the model never saw:** train on the training set, choose settings on the validation set, report once on the test set.

Example 1.1 — classify the problem

Problem	Type	One training row
Predict tomorrow's peak electricity demand in Dehradun	supervised, regression	one day's weather and calendar features + that day's peak demand
Group 50,000 shoppers by purchasing behaviour, no categories given	unsupervised (clustering)	one customer's purchase summary, no target
Pneumonia or not, from 40,000 labelled X-rays	supervised, classification	one X-ray's pixels + the radiologist's label
Robot arm placing a cup: +1 success, -1 drop	reinforcement	no table — a stream of (state, action, reward)
Predict a student's end-sem <i>percentage</i>	supervised, regression	attendance, internal marks, ... + the percentage
Predict the student's <i>letter grade</i> A–F	supervised, classification	the same features + the grade

The last two rows are the same situation with the target recorded differently — which is exactly why the target column decides the type.

T , E , P for fraud detection. T : label each card transaction fraudulent or legitimate. E : past transactions with confirmed outcomes. P : recall on the fraud class together with the false-alarm rate. *Not* plain accuracy: if 0.1% of transactions are fraud, a model that says “legitimate” every time is 99.9% accurate and catches nothing.

Example 1.2 — 1-nearest neighbour, and reading results

1-NN by hand. Five training flowers (petal length, petal width): (1.4, 0.2) setosa, (1.3, 0.2) setosa, (4.7, 1.4) versicolor, (4.5, 1.5) versicolor, (6.0, 2.5) virginica. New flower (4.6, 1.4). Distances $\sqrt{\Delta_1^2 + \Delta_2^2}$: 3.42, 3.51, **0.10**, 0.14, 1.78. The nearest is (4.7, 1.4), so 1-NN predicts **versicolor**. It has no training step: it stores the data and does all its work at prediction time, and it rests entirely on distance — which is why feature scaling (Section 11)

matters to it.

Reading train/test scores. Three models on the same data:

	A	B	C
training R^2	0.62	0.88	0.999
test R^2	0.60	0.85	0.41
diagnosis	underfitting	good fit	overfitting

A: both scores low and close — too simple; add features or flexibility. C: near-perfect on training, poor on test — it has memorised. Deploy **B**.

Extrapolation. A line fitted on houses of 1000–3000 sq ft is $\hat{y} = -50.9 + 0.0754x$ (lakh). At 2400 sq ft: 130.1 lakh. At 400 sq ft: -20.7 lakh — a negative price. Outside the range of the training data the model has no information and gives no warning.

Mistakes that cost marks

- **Deciding the type from the features** instead of the target.
- **A vague P** (“good performance”). Name the measure and say why it suits the task.
- **Quoting a training score as performance.** 100% on the training data is consistent with memorisation. Ask for the test score.
- **Using ML where an exact rule exists** (GST on an invoice, a published fee formula): learning an approximation can only be worse.

Exercises 1 — numerical

- E1.1** A model is $\hat{y} = 5 + 0.06x$ with x in sq ft and $\hat{y}$ in lakh. Predict at $x = 1000$.
- E1.2** An input matrix has 120 houses and 4 features. How many numerical feature entries does it hold? What is its shape, and the shape of y ?
- E1.3** Using the five flowers of Example 1.2, predict the species of (1.5, 0.3) by 1-NN. Show all five distances.
- E1.4** A hospital’s readmission model is 94% accurate overall, yet on the 6% of patients who are readmitted it is right only 20% of the time. How can both be true? What measure would have exposed it?

Short-answer questions, with model answers

- C1.1** *Traditional programming vs ML: what is input and what is output?* Traditional: data + hand-written rules in, answers out. ML: data + known answers in, the rules (a model) out.
- C1.2** *Classify: (a) group unlabelled houses by similarity; (b) a robot learning from rewards.* (a) Unsupervised — no feedback at all, only structure. (b) Reinforcement — feedback is a reward for an action, not a correct answer.
- C1.3** *Parameter vs hyperparameter, one example each.* A parameter is learned from data (b_1 in $\hat{y} = b_0 + b_1x$); a hyperparameter is set before learning and never changed by the algorithm (k in k -NN, the learning rate η , a polynomial degree).
- C1.4** *Why is unsupervised learning harder to evaluate?* There is no correct answer to subtract the prediction from, so it is judged by stability, internal separation measures and usefulness — all of which need judgement, not arithmetic.
- C1.5** *“fit” vs “predict”; which data for what?* **fit** learns parameters from the training set; **predict** applies them to new rows. Training set: fit. Validation set (or CV on training data): choose hyperparameters. Test set: one final, honest report.
- C1.6** *Removing a caste or religion column — does the model then ignore it?* No. Local-

ity, surname, school or pincode act as proxies, and a flexible model reconstructs the attribute from them; removing the column only removes your ability to audit the effect.

C1.7 *Roles of NumPy, pandas and scikit-learn.* NumPy: fast vectorised arithmetic on arrays. pandas: loading, cleaning and grouping tables. scikit-learn: one `fit/predict` interface for preprocessing, models and evaluation.

C1.8 *What is wrong with each line?* `model.fit(X_test, y_test)` — fitting on the test set destroys it as a test; fit on the training data. `model.fit(df["Area"], df["Price"])` — X must be 2-D: `df[["Area"]]`. `X_train, X_test = train_test_split(X, y, ...)` — with X and y it returns four objects. `model.score(X_train, y_train)` — a training score measures fit, not performance; score on the test set.

Unit II — Loss functions (L2): the regression losses

2 Regression losses: SSE, MSE, RMSE, MAE and Huber

The method: computing a regression loss

1. Write the residual row $e_i = y_i - \hat{y}_i$. Keep the sign: it tells you over- or under-prediction, and Huber needs $|e_i|$.
2. One column per loss: e_i^2 (SSE, MSE, RMSE), $|e_i|$ (MAE), and for Huber first compare $|e_i|$ with δ , then pick the branch:

$$L_\delta(e) = \begin{cases} \frac{1}{2}e^2 & |e| \leq \delta \\ \delta(|e| - \frac{1}{2}\delta) & |e| > \delta \end{cases}$$

3. $\text{SSE} = \sum e_i^2$; $\text{MSE} = \text{SSE}/n$; $\text{RMSE} = \sqrt{\text{MSE}}$ (back in the units of y); $\text{MAE} = \frac{1}{n} \sum |e_i|$; mean Huber $= \frac{1}{n} \sum L_\delta(e_i)$.
4. Comment on which row dominates, and on robustness: squaring makes a doubled residual cost four times as much.
5. **Gradients** (with respect to the prediction $\hat{y}$): $e^2 \rightarrow -2e$; $\frac{1}{2}e^2 \rightarrow -e$; $|e| \rightarrow -\text{sign}(e)$ for $e \neq 0$; Huber $\rightarrow -e$ inside, $-\delta \text{sign}(e)$ outside.

Example 2.1 — three houses, every regression loss

Observed $y = (5, 5, 5)$, predicted $\hat{y} = (4, 7, 2)$, Huber $\delta = 1$.

house	1	2	3	Σ
$e = y - \hat{y}$	1	-2	3	2
meaning	under	over	under	
e^2	1	4	9	14
$ e $	1	2	3	6
$ e \leq 1?$	yes	no	no	
Huber	$\frac{1}{2}(1)^2 = 0.5$	$1(2 - 0.5) = 1.5$	$1(3 - 0.5) = 2.5$	4.5

$$\text{SSE} = 14, \quad \text{MSE} = \frac{14}{3} = 4.6667, \quad \text{RMSE} = 2.1602,$$

$$\text{MAE} = \frac{6}{3} = 2, \quad \text{Huber} = \frac{4.5}{3} = 1.5.$$

The mean *signed* residual, $2/3$, is useless as an error measure: residuals $(3, -3)$ would average 0 while both miss by 3. If y is in lakh, MSE is in lakh²; RMSE and MAE are in lakh.

Huber is smooth at the join. At $e = \delta$ both branches give $\frac{1}{2}\delta^2$, and both slopes equal δ . δ is measured in the units of y and sets where the linear tail begins.

Example 2.2 — the best constant: MSE picks the mean, MAE the median

Targets (0, 0, 9); the model must predict one constant w .

constant	SSE	sum of $ e $
$w = 3$ (mean)	$9 + 9 + 36 = \mathbf{54}$	$3 + 3 + 6 = 12$
$w = 0$ (median)	$0 + 0 + 81 = 81$	$0 + 0 + 9 = \mathbf{9}$

Squared error is minimised by the mean (3); absolute error by the median (0). The single extreme target drags the MSE answer; the MAE answer stays with the typical value.

From loss to gradient. Constant w , targets (1, 3), row loss $\frac{1}{2}(w - y)^2$, at $w = 4$: row losses 4.5 and 0.5, so $J = 2.5$; row gradients $w - y = 3$ and 1, so $J'(4) = 2$. Loss and gradient are different quantities — the gradient is what the optimiser in Unit III uses. Minimising SSE or MSE gives the same minimiser; their gradients differ by the factor n , which matters for the learning rate.

Mistakes that cost marks

- **Averaging signed residuals** as an error measure.
- **Huber's linear branch without** $-\frac{1}{2}\delta$: it is $\delta(|e| - \frac{1}{2}\delta)$, not $\delta|e|$.
- **Choosing the Huber branch from e instead of $|e|$** : $e = -2$ with $\delta = 1$ is *outside*.
- **MSE in the wrong units.** Report RMSE when the units matter.

Exercises 2 — numerical

- E2.1** $y = 50$, $\hat{y} = 47$: the residual? And for $y = 40$, $\hat{y} = 45$: is the model over- or under-predicting?
- E2.2** Residuals $(-1, 2, 4)$: compute MSE, MAE and mean Huber loss with $\delta = 1$.
- E2.3** With $\delta = 2$ and residuals $(1, -3)$, compute the mean Huber loss.
- E2.4** Errors $(-4, 2)$: MAE and MSE. A residual changes from 3 to 6: by what factor does its squared penalty change? Its absolute penalty?
- E2.5** $y = (10, 12, 15, 20)$, $\hat{y} = (11, 12, 13, 24)$. Compute MSE, MAE, RMSE and Huber ($\delta = 2$). Which row supplies most of the MSE?
- E2.6** For $y = (1, 2, 2, 3, 12)$, find the best constant under MSE and under MAE. The 12 is mistyped as 112: what happens to each?

Short-answer questions, with model answers

- C2.1** *Is a loss function the same as an optimiser?* No. The loss states the penalty to be minimised; the optimiser is the rule that changes the parameters to reduce it.
- C2.2** *Show that $RMSE \geq MAE$.* $\frac{1}{n} \sum e_i^2 - (\frac{1}{n} \sum |e_i|)^2$ is the variance of the numbers $|e_i|$, which is ≥ 0 ; so $MSE \geq MAE^2$. Equality only when every $|e_i|$ is the same.
- C2.3** *When would you choose MAE or Huber over MSE?* When a few large residuals (outliers, or a heavy-tailed target) should not dominate the fit. MSE suits cases where a large miss really is disproportionately costly.
- C2.4** *Why does MAE have no ordinary derivative at $e = 0$?* $|e|$ has a corner there; any value in $[-1, 1]$ is a valid subgradient. Huber is differentiable everywhere, including at $\pm\delta$.
- C2.5** *Mean loss vs summed loss — same training?* Same minimiser on a fixed dataset, but

the gradient scales with n , so with a fixed learning rate (or an unscaled regularisation weight) the training behaves differently.

Unit III — Optimizer functions (L3–L4)

3 Gradient descent by hand

The method: running gradient descent on paper

1. Write the objective and differentiate it. For the toy problem, row gradients are $g_i = w - y_i$ and the full gradient is their mean, $J'(w) = w - 2$. For the MSE of a line $\hat{y} = b + wx$, $\partial J / \partial w = \frac{2}{n} \sum (\hat{y}_i - y_i)x_i$ and $\partial J / \partial b = \frac{2}{n} \sum (\hat{y}_i - y_i)$.
2. Update: $w_{\text{new}} = w_{\text{old}} - \eta J'(w_{\text{old}})$. Subtracting a positive gradient moves left; a negative one moves right.
3. Tabulate k , w_k , $J(w_k)$, $J'(w_k)$ — one row per step — and check the loss falls.
4. **Stability.** For the toy problem $w_{\text{new}} - 2 = (1 - \eta)(w_{\text{old}} - 2)$: the error shrinks when $|1 - \eta| < 1$, i.e. $0 < \eta < 2$. In general, for $J = \frac{a}{2}(w - c)^2$ the factor is $1 - a\eta$ and the range is $0 < \eta < 2/a$; for $(w - c)^2$ (no half) it is $1 - 2\eta$ and $0 < \eta < 1$.

Example 3.1 — the toy problem, and four learning rates

Targets 1 and 3, start $w_0 = 6$, $\eta = 0.2$. At $w = 6$: row losses $\frac{1}{2}5^2 = 12.5$ and $\frac{1}{2}3^2 = 4.5$, $J = 8.5$; row gradients 5 and 3, mean 4.

	k	w_k	$J(w_k)$	$J'(w_k) = w_k - 2$
	0	6.0000	8.5000	4.0000
	1	5.2000	5.6200	3.2000
	2	4.5600	3.7768	2.5600
	3	4.0480	2.5972	2.0480
	4	3.6384	1.8422	

The minimum is at $w = 2$ with $J(2) = 0.5$ — not zero, because the two targets disagree.

η	$1 - \eta$	first steps from 6	behaviour
0.2	0.8	5.2, 4.56, 4.048	approaches gradually
1	0	2	reaches the optimum in one step
1.2	-0.2	1.2, 2.16, 1.968	overshoots, but converges
2	-1	-2, 6, -2	undamped oscillation

Example 3.2 — two parameters: a line through three points

Fit $\hat{y} = b + wx$ to $x = (1, 2, 3)$, $y = (3, 5, 7)$ with $J = \frac{1}{3} \sum (\hat{y}_i - y_i)^2$, from $(b, w) = (0, 0)$, $\eta = 0.1$.

Step 1. Errors $\hat{y} - y = (-3, -5, -7)$, $J = 27.6667$.

$$\frac{\partial J}{\partial b} = \frac{2}{3}(-15) = -10, \quad \frac{\partial J}{\partial w} = \frac{2}{3}(-3 - 10 - 21) = -22.6667,$$

so $b = 1.0000$, $w = 2.2667$. **Step 2.** Errors $(0.2667, 0.5333, 0.8000)$, $J = 0.3319$; gradient $(1.0667, 2.4889)$; so $b = 0.8933$, $w = 2.0178$ and $J = 0.0053$. The true line is $y = 1 + 2x$. Both partial derivatives are computed from the *same* old parameters before either is updated.

Mistakes that cost marks

- **Adding the gradient** instead of subtracting it.
- **Confusing the loss value with the gradient.** At $w = 6$, $J = 8.5$ but $J' = 4$.
- **Dropping or doubling the factor 2.** With $\frac{1}{2}(w - y)^2$ the row gradient is $w - y$; with $(w - y)^2$ it is $2(w - y)$.
- **“Larger η is faster.”** $\eta = 0.8$ and $\eta = 1.2$ have the same $|1 - \eta|$ and converge at the same rate.

Exercises 3 — numerical

- E3.1** For $J(w) = \frac{1}{2}(w - 3)^2$: find $J'(5)$, and the next parameter from $w = 5$ with $\eta = 0.1$.
- E3.2** For that objective, derive the range of constant learning rates that converge to 3 from any start, and the rate that gets there in one step.
- E3.3** Toy problem (targets 1, 3): at $w = 1$, find the full gradient and the direction of the step. From $w = 5$ with $\eta = 0.1$, take one step.
- E3.4** Five steps of gradient descent on $f(w) = (w + 1)^2$ from $w_0 = 3$ with $\eta = 0.2$. By what factor does the distance to the minimum shrink?
- E3.5** Fit $\hat{y} = b + wx$ to $x = (0, 1, 2)$, $y = (1, 3, 5)$ with $J = \frac{1}{3} \sum (\hat{y} - y)^2$, from $(0, 0)$, $\eta = 0.1$. Two steps; report J after each.

4 Batch, stochastic and mini-batch gradient descent

The method: updates, batches and epochs

1. An **update** is one change to the parameters; a **batch** is the set of rows that supplied its gradient; an **epoch** is one pass through the training data.
2. **Full batch**: one update per epoch, the mean gradient over all rows. **SGD**: one row per update. **Mini-batch**: B rows per update, averaged over the batch’s actual size.
3. Updates per epoch = $\lceil n/B \rceil$ when the last short batch is kept.
4. In SGD each row’s gradient is taken at the *current* parameter, the one just produced — not at the value the epoch started with.

Example 4.1 — SGD and mini-batch on the toy problem

Start $w = 6$, $\eta = 0.2$, rows visited in the order 1, 3, 1, 3.

t	selected y	w before	$g = w - y$	w after
1	1	6.0000	5.0000	5.0000
2	3	5.0000	2.0000	4.6000
3	1	4.6000	3.6000	3.8800
4	3	3.8800	0.8800	3.7040

Mini-batch with both rows: gradient 4 $\Rightarrow w = 5.2$; then 3.2 $\Rightarrow w = 4.56$. (Here the batch is the whole toy dataset, so it is also full-batch descent.)

SGD can increase the full loss. At the optimum $w = 2$ the full gradient is 0, but the row gradients are +1 and −1. One SGD step on the $y = 1$ row moves w to $2 - 0.2(1) = 1.8$ and the full loss rises from 0.5 to 0.52. That is sampling noise, not a bug — and why constant-rate SGD fluctuates around the optimum.

Example 4.2 — one epoch of each, and counting updates

$x = (1, 2, 3, 4)$, $y = (3, 5, 8, 9)$, model $\hat{y} = wx$, row loss $(wx_i - y_i)^2$ with gradient $2x_i(wx_i - y_i)$, $w = 0$, $\eta = 0.02$. Exact answer $w^* = \sum xy / \sum x^2 = 73/30 = 2.4333$.

- **Batch:** gradients at 0 are $(-6, -20, -48, -72)$, mean -36.5 ; $w = 0.73$.
- **SGD:** 0.12, 0.5008, 1.2805, **1.9010**.
- **Mini-batch** $B = 2$: batch $\{1, 2\}$ mean -13 , $w = 0.26$; batch $\{3, 4\}$ at 0.26: -43.32 and -63.68 , mean -53.5 , $w = \mathbf{1.33}$.

Same work (four row-gradients), three answers: more updates per epoch mean later gradients are taken at better parameters.

Counting. 100 rows, $B = 20$: 5 updates per epoch. 103 rows, $B = 20$, last batch kept: $\lceil 103/20 \rceil = 6$ per epoch, 24 in four epochs. $n = 12,000$: $B = 1 \rightarrow 12,000$ updates, $B = 32 \rightarrow 375$, $B = n \rightarrow 1$ — and every one of them uses 12,000 row-gradients per epoch.

Mistakes that cost marks

- **Evaluating every SGD gradient at the starting w .**
- **Summing a mini-batch gradient** instead of averaging it.
- **“Epoch” = “update”.** An epoch contains $\lceil n/B \rceil$ updates.
- **Shuffling time-series rows** as if they were independent.

Exercises 4 — numerical

- E4.1** 100 training rows, batch size 20, no rows dropped: updates per epoch?
- E4.2** Constant model $w = 4$, targets $(1, 3)$, row loss $\frac{1}{2}(w - y)^2$: the batch-mean gradient?
- E4.3** At $w = 2$ with targets $(1, 3)$, compare the full-data gradient with the single-row SGD gradient for $y = 1$. Can that SGD step increase the full loss?
- E4.4** Toy problem from $w = 6$, $\eta = 0.2$: if the first selected target were 3 rather than 1, where would the first update land?
- E4.5** $x = (2, 1, 3)$, $y = (4, 3, 5)$, $\hat{y} = wx$, $w_0 = 0$, $\eta = 0.05$. One batch step and one SGD epoch (in the order given). Which ends nearer w^* ?
- E4.6** $n = 50,000$, $B = 250$, 10 epochs: how many updates, and how many row-gradient evaluations?

5 Momentum

The method: momentum updates by hand

1. $v_t = \beta v_{t-1} + g_t$, then $w_t = w_{t-1} - \eta v_t$, from $v_0 = 0$.
2. Tabulate t , g_t (at the current w), v_t , w_t .
3. Facts: $v_t = \sum_j \beta^j g_{t-j}$, a decaying sum of past gradients; with a constant gradient g , $v_t = g(1 - \beta^t)/(1 - \beta) \rightarrow g/(1 - \beta)$, so the effective step is $\eta/(1 - \beta)$ — ten times η at $\beta = 0.9$.
4. The other convention, $v = \beta v + (1 - \beta)g$, tends to g instead. Never mix them.

Example 5.1 — history matters

One step. $v_{\text{old}} = 2$, $g = -1$, $\beta = 0.9$, $\eta = 0.1$, $w_{\text{old}} = 5$: $v_{\text{new}} = 0.9(2) + (-1) = 0.8$, and $w_{\text{new}} = 5 - 0.1(0.8) = 4.92$. The current gradient says “go right”, but the stored velocity is still positive, so w moves left.

Constant gradient $g = 2$, $\beta = 0.9$: $v = 2, 3.8, 5.42, 6.878, 8.1902, \dots = 2(1-0.9^t)/0.1 \rightarrow 20$. **Alternating** $g = 2, -2, 2, -2$: $v = 2, -0.2, 1.82, -0.362$ — the contributions cancel. In a ravine the along-floor gradient is steady and the across-wall gradient flips, so momentum speeds up along the floor and damps the zig-zag.

Example 5.2 — four steps, two values of β

$f(w) = (w - 2)^2$, $g = 2(w - 2)$, $w_0 = 6$, $\eta = 0.1$.

t	g_t	$v_t = 0.5v_{t-1} + g_t$	w_t
1	8.0000	8.0000	5.2000
2	6.4000	10.4000	4.1600
3	4.3200	9.5200	3.2080
4	2.4160	7.1760	2.4904

Plain GD reaches 3.6384 after four steps. With $\beta = 0.9$: $v = 8, 13.6, 15.92, 14.824$ and $w = 5.2, 3.84, 2.248, \mathbf{0.7656}$ — it has overshoot the minimum, carried by nine tenths of its past velocity although the gradient at step 4 is only 0.496.

Mistakes that cost marks

- **Using g instead of v** in the parameter update.
- **Mixing conventions** ($\beta v + g$ versus $\beta v + (1 - \beta)g$).
- **Calling an overshoot divergence.** Divergence means the swings grow.

Exercises 5 — numerical

- E5.1** $v_0 = 0$, $\beta = 0.9$, $g_1 = g_2 = 2$: find v_1 and v_2 .
- E5.2** With $v_0 = 0$ and a constant gradient $g = 2$, derive v_t for $\beta = 0.9$ and its limit.
- E5.3** Three momentum steps on $f(w) = (w - 3)^2$ from $w_0 = 0$, $\eta = 0.1$, $\beta = 0.5$. Compare with plain GD (1.4640 after three steps).
- E5.4** $\beta = 0.8$; gradients arrive as 4, 3, -1, -2. Give the velocity after each.

Short-answer questions, with model answers

- C5.1** *What does v store?* A decaying (unnormalised) sum of past gradients — the accumulated direction of travel.
- C5.2** *What problem does momentum solve, and what does it cost?* It accelerates along directions where the gradient is consistent and damps oscillation where it flips; the price is overshoot, and one more hyperparameter β .
- C5.3** *Why does the convention matter for the learning rate?* With $v = \beta v + g$ the steady step is $\eta g / (1 - \beta)$; with $v = \beta v + (1 - \beta)g$ it is ηg . The same η gives steps ten times apart at $\beta = 0.9$.

6 Adagrad and RMSprop

The method: adaptive step sizes by hand

1. One accumulator per parameter, starting at 0.
2. **Adagrad** adds: $G_t = G_{t-1} + g_t^2$. **RMSprop** averages: $V_t = \rho V_{t-1} + (1 - \rho)g_t^2$ ($\rho = 0.9$ unless told otherwise).
3. Step: $w_t = w_{t-1} - \eta g_t / (\sqrt{G_t} + \varepsilon)$ (or $\sqrt{V_t}$). Update the accumulator *first*, then step.

Ignore ε by hand unless the accumulator is 0.

4. The numerator keeps the sign of g ; only the denominator is squared.
5. Checks: Adagrad's first step has size exactly η ; RMSprop's is $\eta/\sqrt{1-\rho}$ (3.1623η at $\rho = 0.9$).

Example 6.1 — single steps, and three Adagrad steps

Adagrad, one step. $G_{\text{old}} = 9$, $g = 4$, $\eta = 0.1$, $w_{\text{old}} = 2$: $G_{\text{new}} = 9 + 16 = 25$, $\sqrt{G} = 5$, step $= 0.1 \times 4/5 = 0.08$, $w_{\text{new}} = 1.92$. **RMSprop, one step.** $V_{\text{old}} = 4$, $\rho = 0.9$, $g = 2$, $\eta = 0.1$, $w_{\text{old}} = 5$: $V_{\text{new}} = 0.9(4) + 0.1(4) = 4$, step $= 0.1 \times 2/2 = 0.1$, $w_{\text{new}} = 4.9$.

Three Adagrad steps on $f(w) = (w - 4)^2$, $g = 2(w - 4)$, $w_0 = 0$, $\eta = 1$:

t	g_t	G_t	$\sqrt{G_t}$	step	w_t
1	-8.0000	64.0000	8.0000	+1.0000	1.0000
2	-6.0000	100.0000	10.0000	+0.6000	1.6000
3	-4.8000	123.0400	11.0923	+0.4327	2.0327

Step 1 is exactly η . Then the steps shrink faster than the gradient, because G only ever grows — Adagrad's weakness on long runs. Its strength: a rarely-active coordinate keeps a small G and so a large step.

Example 6.2 — RMSprop on the same bowl

$w_0 = 0$, $\eta = 0.5$, $\rho = 0.9$.

t	g_t	$V_t = 0.9V_{t-1} + 0.1g_t^2$	$\sqrt{V_t}$	step	w_t
1	-8.0000	6.4000	2.5298	+1.5811	1.5811
2	-4.8377	8.1004	2.8461	+0.8499	2.4310
3	-3.1380	8.2750	2.8766	+0.5454	2.9764

Check: $0.5/\sqrt{0.1} = 1.5811$. Adagrad at the same η reaches only 0.5, 0.8293, 1.0854. RMSprop forgets old gradients (it remembers about $1/(1-\rho) = 10$), so its denominator stays near the size of recent gradients and can *fall*. After k zero gradients V becomes $\rho^k V_0$ — the history changes although w does not move.

Mistakes that cost marks

- **Dividing by G instead of $\sqrt{G}$.**
- **Forgetting $(1-\rho)$ in RMSprop**, or using ρg^2 .
- **Stepping with the old accumulator.** Update G or V first.
- **Losing the sign:** with $G_{\text{old}} = 0$, $g = -3$, $\eta = 0.1$, the change is $-0.1(-3)/3 = +0.1$.

Exercises 6 — numerical

- E6.1** $G_{\text{old}} = 9$, $g = 4$: the new accumulator? Then with $G_{\text{new}} = 25$, $g = 4$, $\eta = 0.1$, $w_{\text{old}} = 2$: w_{new} ?
- E6.2** $V_{\text{old}} = 4$, $\rho = 0.9$, $g = 2$, $\eta = 0.1$, $w_{\text{old}} = 5$: w_{new} by RMSprop. And for $V_{\text{old}} = 0$, $g = 2$: V_{new} and the denominator.
- E6.3** Three Adagrad steps on $f(w) = (w - 1)^2$ from $w_0 = 5$, $\eta = 0.5$.
- E6.4** Gradients 4, 2, 1, 0.5: the Adagrad and the RMSprop ($\rho = 0.9$) accumulators after each, and $g/\sqrt{\cdot}$ for each. Which ratio stays larger, and why?
- E6.5** After k consecutive zero gradients, what is V if it started at V_0 ? Does the parameter move during those updates?

7 Adam, with bias correction

The method: Adam by hand

1. Defaults $\beta_1 = 0.9$, $\beta_2 = 0.999$; $m_0 = v_0 = 0$; t counts updates from 1 and is never reset at an epoch boundary.
2. $m_t = \beta_1 m_{t-1} + (1 - \beta_1)g_t$ (signed history); $v_t = \beta_2 v_{t-1} + (1 - \beta_2)g_t^2$ (squared history).
3. Bias correction: $\hat{m}_t = m_t / (1 - \beta_1^t)$, $\hat{v}_t = v_t / (1 - \beta_2^t)$. Divisors: 0.1 and 0.001 at $t = 1$; 0.19 and 0.001999 at $t = 2$.
4. Step: $w_t = w_{t-1} - \eta \hat{m}_t / (\sqrt{\hat{v}_t} + \varepsilon)$. Carry the *uncorrected* m_t , v_t forward.
5. Check: $\hat{m}_1 = g_1$ and $\hat{v}_1 = g_1^2$, so the first step is exactly $\pm\eta$, whatever the size of g_1 .

Example 7.1 — Adam, two steps from a given gradient sequence

$w_0 = 5$, $\eta = 0.1$, gradients $g_1 = 2$, $g_2 = 1$.

Step 1. $m_1 = 0.1(2) = 0.2$; $v_1 = 0.001(4) = 0.004$; $\hat{m}_1 = 0.2/0.1 = 2$; $\hat{v}_1 = 0.004/0.001 = 4$; direction $2/\sqrt{4} = 1$; $w_1 = 5 - 0.1 = 4.9$.

Step 2 (keep the history). $m_2 = 0.9(0.2) + 0.1(1) = 0.28$; $v_2 = 0.999(0.004) + 0.001(1) = 0.004996$; $\hat{m}_2 = 0.28/0.19 = 1.473684$; $\hat{v}_2 = 0.004996/0.001999 = 2.499250$; direction $1.473684/1.580902 = 0.932180$; $w_2 = 4.9 - 0.0932 = 4.806782$.

The second direction is not simply $\text{sign}(g_2) = 1$: both stored histories still remember $g_1 = 2$.

Example 7.2 — three steps on a bowl, and without the correction

$f(w) = (w - 4)^2$, $w_0 = 0$, $\eta = 0.5$.

t	g_t	m_t	v_t	$\hat{m}_t$	$\sqrt{\hat{v}_t}$	w_t
1	-8.0000	-0.8000	0.064000	-8.0000	8.0000	0.5000
2	-7.0000	-1.4200	0.112936	-7.4737	7.5164	0.9972
3	-6.0057	-1.8786	0.148891	-6.9320	7.0484	1.4889

Steps 0.500, 0.497, 0.492: Adam moves at almost exactly the speed η .

Without bias correction the first step would be $0.5 \times 0.8 / \sqrt{0.064} = 1.5811$ — three times η — because m is shrunk by $1 - \beta_1 = 0.1$ but $\sqrt{v}$ only by $\sqrt{1 - \beta_2} = 0.0316$; the ratio is inflated by $0.1/0.0316 = 3.1623$.

Mistakes that cost marks

- $1 - \beta$ **instead of** $1 - \beta^t$ at step 2 or later.
- **Carrying $\hat{m}$, $\hat{v}$ forward** instead of m , v .
- $v = m^2$. v averages g^2 .
- **Calling v the variance.** It is an uncentred second moment.

Exercises 7 — numerical

E7.1 $t = 1$, $g = 2$, $w_0 = 5$, $\eta = 0.1$, defaults: find w_1 . If instead $g_1 = -2$: $\hat{m}_1$, $\hat{v}_1$ and the signed step?

E7.2 Two Adam steps on $f(w) = (w - 1)^2$ from $w_0 = 3$, $\eta = 0.1$.

E7.3 Show that the corrected first step is $-\eta \text{sign}(g_1)$. What is it for $g_1 = -0.003$, $\eta = 0.01$?

E7.4 Without correction, how large is the first step (as a multiple of η) for $\beta_2 = 0.999$? For $\beta_2 = 0.99$ (with $\beta_1 = 0.9$)?

8 Adadelta, and comparing optimisers

The method: Adadelta by hand

1. Two decaying averages: G of squared gradients and U of squared *updates*, both from 0; $\rho = 0.9$; ε inside both roots.
2. In this order: (i) $G_t = \rho G_{t-1} + (1 - \rho)g_t^2$; (ii) $\Delta w_t = -\frac{\sqrt{U_{t-1} + \varepsilon}}{\sqrt{G_t + \varepsilon}} g_t$; (iii) $w_t = w_{t-1} + \Delta w_t$; (iv) $U_t = \rho U_{t-1} + (1 - \rho)\Delta w_t^2$.
3. The numerator uses the *previous* U_{t-1} — U_t cannot exist until Δw_t does. There is no learning rate in the original form: the ratio has the units of w .

Example 8.1 — one step with history, and the first step from zero

With history. $G_{\text{old}} = 4$, $U_{\text{old}} = 1$, $\rho = 0.9$, $g = 2$, ε ignored: $G = 0.9(4) + 0.1(4) = 4$; ratio $\sqrt{1}/\sqrt{4} = 0.5$; $\Delta w = -0.5 \times 2 = -1$; $U = 0.9(1) + 0.1(1) = 1$.

From zero on $f(w) = (w - 3)^2$, $w_0 = 0$, $\varepsilon = 10^{-6}$: $g_1 = -6$, $G_1 = 0.1 \times 36 = 3.6$, $\sqrt{G_1 + \varepsilon} = 1.8974$; $\sqrt{U_0 + \varepsilon} = \sqrt{10^{-6}} = 0.001$; so $\Delta w_1 = -(0.001/1.8974)(-6) = +0.003162$. Tiny: Adagrad's first step here would be η , Adadelta's is set by ε . Then $U_1 = 0.1(0.003162)^2 = 10^{-6}$, the numerator grows to $\sqrt{2} \times 10^{-6} = 0.001414$, and the steps grow as updates accumulate.

Example 8.2 — what each optimiser stores

Method	State beyond w	Main sensitivity
Plain SGD / batch	none	learning rate, sampling, feature scaling
Momentum	decaying signed sum v	β and the velocity convention
Adagrad	cumulative g^2 per coordinate	ever-growing denominator
RMSprop	decaying mean of g^2	ρ , placement of ε
Adam	m , v and the update count t	bias correction, variant
Adadelta	decaying means of g^2 and Δw^2	update order, ε

A fair comparison keeps the model, loss reduction, data split, preprocessing and metric fixed; gives each optimiser its own tuned learning rate (the same number means different step sizes in different rules); repeats stochastic runs; chooses on validation data; and reports on untouched test data together with the compute used. A lower training loss is not proof of better generalisation.

Exercises 8 — numerical

E8.1 $G_{\text{old}} = 4$, $U_{\text{old}} = 1$, $\rho = 0.9$, $g = 2$, ε ignored: compute Δw .

E8.2 $G_{\text{old}} = 1$, $U_{\text{old}} = 0.25$, $\rho = 0.9$, $g = -2$, ε ignored: compute G , Δw and the new U .

Short-answer questions, with model answers

C8.1 What do Adam's m and v average? m : the signed gradients (direction, as in momentum). v : the squared gradients (scale, as in RMSprop).

C8.2 Why can Adagrad learn rare coordinates differently, and why can it stall? Each coordinate has its own G ; rarely active ones keep a small G and a large step. But G never decreases, so every step shrinks towards zero on long runs.

C8.3 What distinguishes RMSprop from Adagrad? An exponentially weighted *average* of g^2 instead of a cumulative sum, so old gradients lose weight and the denominator can fall.

- C8.4** Which two moving averages does Adadelata track? $E[g^2]$ and $E[\Delta w^2]$.
- C8.5** Two correct demonstrations give different trajectories — why? Different loss reduction (sum or mean, with or without $\frac{1}{2}$), momentum convention, ε placement, initial values or learning rate. Compare conventions before declaring an error.
- C8.6** Your analytic gradient is twice the finite-difference value. What do you check first? Whether the loss has a $\frac{1}{2}$ (or an averaging factor) that the derivative dropped.

Unit IV — Data preprocessing (L5–L7)

The housing dataset used in Unit IV

Twelve houses offered for sale in Dehradun (from the course’s chapter notes). Area in sq ft, Age in years, Dist in km to the city centre, buyer Income and Price in lakh rupees.

ID	Area	Bed	Bath	Age	Location	Garage	Condition	Dist	Income	Price
H01	1000	2	1	12	Clement Town	No	Average	8.5	9.5	42
H02	1500	3	2	5	Rajpur Road	Yes	Good	3.2	18	78
H03	?	3	2	9	Sahastradhara	No	Average	8.0	14	61
H04	2000	4	3	2	Rajpur Road	Yes	Excellent	2.5	32	120
H05	1200	2	1	18	Prem Nagar	No	Poor	12.4	7	38
H06	2500	4	3	7	Rajpur Road	Yes	Good	4.0	28	140
H07	1800	3	2	?	Clement Town	Yes	Good	7.8	16	72
H08	3000	5	4	3	Sahastradhara	Yes	Excellent	5.5	45	185
H09	1350	3	1	15	Prem Nagar	?	Average	13.0	11	46
H10	2200	4	3	6	Clement Town	Yes	Good	6.5	24	105
H11	9500	6	6	4	Rajpur Road	Yes	Excellent	2.0	95	620
H12	1600	3	2	150	Prem Nagar	No	Average	10.2	?	58

Seven problems hide in it: three blanks (H03 Area, H07 Age, H09 Garage) and a fourth (H12 Income); an impossible value (H12 Age = 150); a genuine extreme (H11); columns on wildly different scales; three text columns; an identifier (ID) that carries no information; and Income, which is recorded *after* the sale — a leak (Section 16).

9 Missing data

The method: handling a blank

1. **Ask why it is missing.** MCAR: unrelated to anything. MAR: depends on other *observed* columns. MNAR: depends on the missing value itself — cannot be detected from the data, only from how it was collected.
2. **Fix known errors first**, then compute any statistic you will fill with.
3. **Choose:** delete rows (tiny fraction, MCAR only); delete a column (mostly empty); impute the *median* for a skewed numeric column, the *mean* for a symmetric one, the *mode* for a categorical one — group-wise when a sensible group exists; add a *missing-indicator* column when missingness may itself be informative (MNAR); or a model-based imputer (KNN, iterative).
4. **Compute from observed values only**, on the **training** rows only.
5. Know the side effect: mean imputation leaves the mean unchanged but shrinks the spread — the sample variance is multiplied by $(m - 1)/(n - 1)$ when m of n values are observed.

Example 9.1 — the blanks in the housing table

Deletion. Blanks in H03, H07, H09 and H12: $12 \rightarrow 8$ rows, 66.7% kept — a third of the data thrown away to repair four cells of 132.

H03 Area (numeric). Eleven observed values, sum 27,650: mean = $27650/11 = 2513.64$. Sorted: 1000, 1200, 1350, 1500, 1600, **1800**, 2000, 2200, 2500, 3000, 9500; median = 6th value = 1800. The single 9500 pulls the mean 40% above the median; nine of eleven houses are below the mean. Fill with the **median, 1800** (also plausible for a 3-bed, 2-bath house).

H09 Garage (categorical). Global mode: Yes (7) vs No (4) $\Rightarrow$ Yes. But the other Prem Nagar houses (H05, H12) have no garage: the **group-wise mode** within Location says No — more likely correct.

H07 Age, and the order of operations. With H12's 150 still in: mean $231/11 = 21.00$, median 7. After correcting $150 \rightarrow 15$: mean $96/11 = 8.73$, median 7. Impute before fixing the error and H07 is recorded as 21 years old; the other Clement Town houses are 12 and 6. Fix errors first — and note that the median did not move.

H12 Income, with an indicator. Fill with the median (18) and add `Income_missing = 1`: a column is added, no row is. (Income is later removed as a leak — Section 16.)

Example 9.2 — mean imputation shrinks the spread

Marks 40, 46, ?, 50, 54, ?, 60, 98. Observed ($m = 6$): mean $348/6 = 58$, median $(50 + 54)/2 = 52$, sample sd 20.7461. Right-skewed (mean > median) $\Rightarrow$ the median is the safer fill.

If both holes are filled with the mean 58: the mean stays 58, the sum of squared deviations is unchanged, but n grows from 6 to 8, so $s_{\text{after}}^2 = \frac{5}{7}s_{\text{before}}^2$ and $s_{\text{after}} = 20.7461 \times 0.8452 = 17.5336$. The model sees a spread 15% narrower than the real one, and the column's relationship with the target is weakened.

Mistakes that cost marks

- **Counting the blank** in n .
- **Imputing before correcting known errors.**
- **Computing the fill value on all rows** before splitting — leakage.
- **Filling a blank that is not missing**: an empty `Garage_Area` may simply mean 0.

Exercises 9 — numerical

- E9.1** Training values 10, 20, ?, 40. Impute with the observed median.
- E9.2** A column holds 1200, 1500, ?, 1800, 2000, 2500, 9000. Mean and median of the observed values; which do you impute with?
- E9.3** The eleven observed buyer incomes are 7, 9.5, 11, 14, 16, 18, 24, 28, 32, 45, 95. Mean and median; which fills H12's blank, and why?
- E9.4** $n = 100$ with 20 values mean-imputed. By what factor does the population sd shrink? The sample sd?

Short-answer questions, with model answers

- C9.1** *Why is the median usually safer than the mean? When is the mean as good?* The mean uses magnitudes, so one extreme value moves it without limit; the median uses ranks. For a roughly symmetric column the two nearly coincide.
- C9.2** *Distinguish MCAR, MAR and MNAR. Which cannot be detected from the data?* See the method box. MNAR cannot, because the evidence needed is exactly what is missing; handle it with an indicator column and state the limitation.
- C9.3** *Missing income is more common among high earners. Deletion or median imputation?* This is MNAR. Deleting drops the high earners and biases the data; plain median imputation understates their income. Impute the median *and* add `Income_missing`,

fitted on training data only, and report the limitation.

C9.4 *Is every unusual house price a data error?* No — see Section 10.

10 Outliers and transformations

The method: detecting and treating an extreme value

1. **Sort**, find Q_1 and Q_3 (hinges: medians of the lower and upper halves), $IQR = Q_3 - Q_1$.
2. **Tukey fences**: flag values below $Q_1 - 1.5IQR$ or above $Q_3 + 1.5IQR$.
3. **z -score**: $z = (x - \bar{x})/s$ (sample s), flag $|z| > 3$. Beware *masking* — the outlier inflates the $\bar{x}$ and s it is judged by — and the ceiling $|z| \leq (n - 1)/\sqrt{n}$.
4. **Modified z** : $M = 0.6745(x - \tilde{x})/\text{MAD}$ with $\text{MAD} = \text{median } |x_i - \tilde{x}|$, flag $|M| > 3.5$.
5. **Decide**, don't delete by reflex. Impossible $\Rightarrow$ correct it or treat as missing. Genuine $\Rightarrow$ keep it and use a tolerant method (log transform, robust scaler, MAE/Huber, trees), clip (winsorise) at the fence, or scope the model and say so.
6. **Shape**: skewness $g_1 = m_3/m_2^{3/2}$ with $m_k = \frac{1}{n} \sum (x_i - \bar{x})^k$; $g_1 > 0$ means a long right tail — a candidate for $\ln x$.

Example 10.1 — the twelve prices

Sorted: 38, 42, 46, 58, 61, 72 | 78, 105, 120, 140, 185, 620.

IQR. $Q_1 = (46 + 58)/2 = 52$, $Q_3 = (120 + 140)/2 = 130$, $IQR = 78$. Fences $52 - 117 = -65$ and $130 + 117 = 247$. $620 > 247$: **H11 flagged**. H08 (185) is not — the rule picks out a separate observation, not merely the top of the range. (NumPy's interpolated quartiles are 55 and 125, upper fence 230: the same verdict.)

z -score. $\bar{x} = 130.42$, $s = 160.46$: $z = (620 - 130.42)/160.46 = 3.05$ — only just over 3, because H11 itself inflated s from 46.63 to 160.46. Judged against the other eleven houses ($\bar{x} = 85.91$, $s = 46.63$) it scores $z = 11.45$. And with $n = 12$ no point can exceed $11/\sqrt{12} = 3.175$: the $|z| > 3$ rule could never fire on ten points or fewer.

Decision. H11 is a genuine luxury property: keep it if you will price such houses, or restrict the model's stated domain and say so. H12's Age = 150 is impossible for its context: correct it to 15. **Treatments.** Clipping sets $620 \rightarrow 247$ (a fiction, but capped). $\ln$: prices 38–620 (ratio 16.3 $\times$) become 3.64–6.43 (ratio 1.77 $\times$); nothing is deleted, the scale is changed, and predictions must be exponentiated back.

Example 10.2 — skewness, and why the log helps

$x = (1, 1, 2, 4, 7)$, $\bar{x} = 3$, deviations $(-2, -2, -1, 1, 4)$: $m_2 = 26/5 = 5.2$, $m_3 = (-8 - 8 - 1 + 1 + 64)/5 = 9.6$, $g_1 = 9.6/5.2^{1.5} = 9.6/11.8579 = +0.8096$ — a right tail, produced by the single 7.

$\ln$ of the geometric sequence 1, 4, 16, 64 gives 0, 1.3863, 2.7726, 4.1589: equal gaps of $\ln 4$. The log turns “four times bigger” into “one step bigger”, which is why prices, areas and incomes usually want it.

Mistakes that cost marks

- “**Outlier = error.**” H11 is rare but true.
- **Concluding “no outliers” from $|z| \leq 3$ on a small sample.**
- **Recomputing the fences on the test set.** Q_1 , Q_3 and the fences are learned from training data and applied unchanged.

- **Deleting rows from the test set.** It must reflect reality.

Exercises 10 — numerical

- E10.1** $Q_1 = 10$, $Q_3 = 18$: the IQR and the upper fence.
E10.2 Apply the IQR rule to 20, 25, 30, 35, 40, 45, 50, 200: Q_1 , Q_3 , both fences, flagged values.
E10.3 12, 14, 15, 15, 16, 17, 18, 19, 60 ($n = 9$). Apply the IQR rule (leave the median out of both halves), the z -score rule and the modified z to the 60. Why does the z rule fail?
E10.4 The largest possible $|z|$ for $n = 10$? $n = 11$? For which can $|z| > 3$ ever fire?
E10.5 Skewness g_1 of (0, 0, 0, 0, 10). Compare with the ceiling $(n - 2)/\sqrt{n - 1}$.

Short-answer questions, with model answers

- C10.1** *A student removed every row with $|z| > 3$, then mean-imputed, then split. Three mistakes?* (i) Deleting by a rule instead of deciding error vs genuine (and the z rule is masked on small samples); (ii) mean imputation of skewed columns, computed on all rows; (iii) every statistic was learned before the split — leakage into the test set.
C10.2 *What changes when prices are log-transformed before regression?* The model predicts $\ln(\text{price})$; errors become roughly proportional (percentage) errors; predictions must be exponentiated back, and an RMSE on the log scale is not in lakh.
C10.3 *Where does the 1.5 come from?* A convenient compromise: for normal data the fences cover about 99.3%, so about 0.7% of clean points are flagged. Use 3 to flag only very extreme points.

11 Feature scaling

The method: the four scalers

1. From the **training** column compute the statistics, then apply $x \mapsto (x - a)/b$ to training and test rows:

scaler	a	b	result
min-max (normalisation)	$x_{\min}$	$x_{\max} - x_{\min}$	$[0, 1]$ on training data
standardisation (z)	$\bar{x}$	σ (divisor n)	mean 0, sd 1
robust	median	IQR	median 0, IQR 1
max-abs	0	$\max x $	$[-1, 1]$, zeros kept

2. Test values may land outside $[0, 1]$: that is correct, and it signals a value outside the training range. Never refit on the test set.
3. **Who needs it?** Anything that compares or combines features numerically: k -NN, k -means, SVM (RBF), neural networks, gradient descent, ridge/lasso. Not trees or forests (they split one feature at a time on order), and not least squares by the exact formula.

Example 11.1 — scaling changes the nearest neighbour

Query house (2000 sq ft, 4 bed); $A = (2050, 2)$; $B = (1500, 4)$.

Unscaled: $d(Q, A) = \sqrt{50^2 + 2^2} = 50.04$; $d(Q, B) = \sqrt{500^2 + 0} = 500$. A is “nearest” — the bedroom difference supplies 4 of 2504, 0.16% of the squared distance.

Min-max with Area $\in [1000, 3000]$, Bedrooms $\in [2, 5]$: $Q = (0.500, 0.667)$, $A = (0.525, 0)$, $B = (0.250, 0.667)$; $d(Q, A) = \sqrt{0.000625 + 0.4444} = 0.667$, $d(Q, B) = 0.250$. Now **B is nearest**: a 4-bedroom house is priced from a 4-bedroom house. Scaling added no

information; it stopped one column monopolising the distance.

Min–max is ruined by an outlier. With H11 in the column ($x_{\max} = 9500$), a 2000 sq ft house scales to $1000/8500 = 0.1176$ and a 3000 sq ft house to 0.2353 — every ordinary house is squashed into the bottom quarter.

Example 11.2 — one column through four scalers

$v = (2, 4, 4, 4, 5, 5, 7, 9)$: mean 5, deviations $(-3, -1, -1, -1, 0, 0, 2, 4)$, $\sigma = \sqrt{32/8} = 2$; min 2, max 9; median 4.5; hinges $Q_1 = 4$ (median of 2, 4, 4, 4), $Q_3 = 6$ (median of 5, 5, 7, 9), IQR = 2.

v	2	4	4	4	5	5	7	9
standard $(v - 5)/2$	-1.5	-0.5	-0.5	-0.5	0	0	1	2
min-max $(v - 2)/7$	0	.2857	.2857	.2857	.4286	.4286	.7143	1
robust $(v - 4.5)/2$	-1.25	-0.25	-0.25	-0.25	0.25	0.25	1.25	2.25
max-abs $v/9$	.2222	.4444	.4444	.4444	.5556	.5556	.7778	1

(RobustScaler uses NumPy quartiles, 4 and 5.5, so it divides by 1.5; by hand, state your rule.) **A test value outside the range:** training Area 200–800, test house 950: correctly $(950 - 200)/600 = 1.25$. A scaler wrongly fitted on all the data (max 950) gives 1.00 — hiding exactly the fact that the house lies outside the training range.

Mistakes that cost marks

- **Sample sd in standardisation.** StandardScaler divides by n .
- **fit_transform on the test set.** Fit on training, transform the test set.
- **Scaling for a tree** and claiming it helped.
- **Min–max on a column with kept extremes.** Use the robust scaler.

Exercises 11 — numerical

- E11.1** Training mean 80, sd 20: standardise $x = 100$. Training min 10, max 30: min–max scale a held-out 40 without clipping.
- E11.2** Area ranges from 500 to 5000: min–max scale $x = 2000$.
- E11.3** For $\{10, 20, 30, 40, 50\}$ compute μ , σ (divisor n) and the z -score of 50.
- E11.4** Using the data of E10.2 (median 37.5, IQR = 20), robust-scale $x = 60$.
- E11.5** Query (1800 sq ft, 3 bed); A (1850, 1); B (1400, 3). Distances unscaled, then with Area $\in [1000, 3000]$ and Bedrooms $\in [1, 5]$ min–max scaled. Nearest in each case?
- E11.6** Put $v = (-4, 0, 2, 6, 8, 16)$ through all four scalers (hinges for the robust scaler).

Short-answer questions, with model answers

- C11.1** *Normalisation vs standardisation, one point.* Normalisation (min–max) maps to a range, $[0, 1]$; standardisation maps to mean 0 and sd 1 and is not bounded.
- C11.2** *Why does a decision tree need no scaling?* A split asks “is $x_j < t$?”. Any increasing rescaling moves t but not which rows fall on each side.
- C11.3** *Features range from 0.001 to 5000 and you use an RBF SVM.* Standardise: the kernel uses squared distance, which the large feature would otherwise dominate.

12 Encoding and feature engineering

The method: turning text into numbers, and building better features

1. **Nominal** (no order: Location) $\rightarrow$ one-hot, one 0/1 column per level. With an intercept and interpreted coefficients, drop one level (`drop='first'`) to avoid the dummy-variable trap. Set `handle_unknown='ignore'` for levels unseen in training.
2. **Ordinal** (real order: Condition) $\rightarrow$ integer codes *in that order*, given explicitly: Poor 0, Average 1, Good 2, Excellent 3.
3. **Binary** (Garage) $\rightarrow$ one 0/1 column.
4. **Columns after one-hot**: original columns $-1 + k$ for a column with k levels ($-1 + k - 1$ with drop-first).
5. **Feature engineering**: create columns from domain knowledge — ratios, sums, differences, date parts — *only* from information available at prediction time. A feature computed from the target is a leak.

Example 12.1 — encoding the housing table

Location (4 levels) $\rightarrow$ `Loc_ClementTown`, `Loc_PremNagar`, `Loc_RajpurRoad`, `Loc_Sahastradhara`; H02 (Rajpur Road) becomes (0,0,1,0). With drop-first, 3 columns and Clement Town is the baseline (0,0,0).

Why not codes 0,1,2,3 for Location? They impose an order and equal spacing that do not exist: a linear model must then put Prem Nagar's effect exactly between Clement Town's and Rajpur Road's, and k -NN would call Clement Town "closer" to Prem Nagar than to Sahastradhara. With three colours coded 0, 1, 2 and mean targets 20, 50, 10, the best line through (0, 20), (1, 50), (2, 10) is $\hat{y} = 31.67 - 5x$, with fitted values 31.67, 26.67, 21.67 and SSE = 816.7 (against 866.7 for predicting the overall mean): the codes explain almost nothing. One-hot fits the three means exactly (SSE = 0). (Trees are fine with integer codes.)

Dummy-variable trap. With an intercept and all k dummies, the dummies sum to 1 in every row — the intercept column — so $X^T X$ is singular and the coefficients are not unique. Drop one level (or use ridge/lasso, whose penalty makes the solution unique).

Unseen level. A test house in Mussoorie gets (0,0,0,0) under `handle_unknown='ignore'`: it matches no known locality and receives only the intercept (with drop-first it would be mistaken for the baseline), so its prediction should be flagged as less trustworthy. Label encoding would have crashed or silently assigned a meaningless new code.

Example 12.2 — engineering a feature

H09 (1350 sq ft, 3 bed, 1 bath, 46 lakh) and H02 (1500 sq ft, 3 bed, 2 bath, 78 lakh) look nearly identical on Area and Bedrooms, yet H02 costs 70% more. `Area_per_Bedroom`: 450 vs 500; `Bath_per_Bed`: 0.33 vs 0.67 — the layout information was in the table, but only in the *relationship* between columns.

For a 2400 sq ft house with 3 bedrooms and 2 bathrooms: `Area_per_Bedroom` = 800, `Total_Rooms` = 5. `Age_at_sale` is safe (known when you price the house); `Price_per_sqft` is not — it is computed from the price you are trying to predict.

Exercises 12 — numerical

E12.1 Three known localities, full one-hot: how many indicator columns?

E12.2 A dataset has 500 rows and 6 columns, one categorical with 5 levels. Columns after one-hot encoding it? With drop-first?

E12.3 Codes blue 0, green 1, red 2; mean targets 30, 10, 20. Fit the least-squares line through the three points and give its SSE. What would one-hot give?

Short-answer questions, with model answers

- C12.1** *Blood group (A, B, AB, O) for a linear model? Education (School, Bachelor, Master, PhD) for a linear model?* Blood group is nominal $\Rightarrow$ one-hot. Education has a real order $\Rightarrow$ ordinal codes 0–3 in that order.
- C12.2** *When is label encoding nevertheless fine?* For tree-based models, which only split on order within one column.
- C12.3** *Encoding vs feature engineering in one line.* Encoding translates an existing column into numbers; engineering creates new information from existing columns.
- C12.4** *Why is OrdinalEncoder's default wrong for {low, medium, high}?* It sorts alphabetically — high 0, low 1, medium 2 — scrambling the order. Pass the categories explicitly.

13 Data reduction and PCA

The method: reducing columns, and PCA by hand

- Three kinds of reduction:** *feature selection* keeps a subset of the original columns (filter, wrapper, embedded); *dimensionality reduction* (feature extraction, e.g. PCA) builds new columns from all of them; *sampling* (numerosity reduction) keeps fewer rows. Selection cuts columns; sampling cuts rows.
- PCA:** standardise if the units differ; centre; covariance matrix S ; solve $\det(S - \lambda I) = 0$ (for 2×2 , $\lambda^2 - \text{tr}(S)\lambda + \det S = 0$); unit eigenvectors from $(S - \lambda I)u = 0$.
- Shortcut** for two standardised columns with correlation r : $S = \begin{pmatrix} 1 & r \\ r & 1 \end{pmatrix}$, $\lambda = 1 \pm r$, $u = \frac{1}{\sqrt{2}}(1, \pm 1)$.
- Explained variance $\lambda_k / \sum \lambda$; scores $t = x_c \cdot u$. Check $\sum \lambda = \text{tr } S$, $\prod \lambda = \det S$.

Example 13.1 — five houses, Area and Bedrooms

Area (1000, 1500, 2000, 2500, 3000), Bedrooms (2, 3, 4, 4, 5). **Standardise** (divisor n): Area $\mu = 2000$, $\sigma = 707.11$, $z_A = (-1.4142, -0.7071, 0, 0.7071, 1.4142)$; Bedrooms $\mu = 3.6$, $\sigma = \sqrt{5.2/5} = 1.0198$, $z_B = (-1.5689, -0.5883, 0.3922, 0.3922, 1.3728)$.

Correlation $r = \frac{1}{5} \sum z_A z_B = 4.8536/5 = 0.9707$, so $S = \begin{pmatrix} 1 & 0.9707 \\ 0.9707 & 1 \end{pmatrix}$. **Eigenvalues** $\lambda_1 = 1.9707$, $\lambda_2 = 0.0293$ (sum 2); PC1 explains $1.9707/2 = 98.5\%$. **Directions** $u_1 = \frac{1}{\sqrt{2}}(1, 1)$ (“overall size”), $u_2 = \frac{1}{\sqrt{2}}(1, -1)$. **Scores** PC1 = $(z_A + z_B)/\sqrt{2}$: H01 $(-1.4142 - 1.5689)/1.4142 = -2.1094$; all five: $-2.1094, -0.9160, 0.2774, 0.7774, 1.9707$. Two columns replaced by one, 1.5% of the variance lost — because the two columns said nearly the same thing.

Example 13.2 — a covariance matrix that needs the full method

Points (1, 1), (3, 3), (4, 3), (5, 5), (7, 3); mean (4, 3); centred $(-3, -2), (-1, 0), (0, 0), (1, 2), (3, 0)$. Sums $\sum x_1^2 = 20$, $\sum x_2^2 = 8$, $\sum x_1 x_2 = 8$; divide by $n - 1 = 4$: $S = \begin{pmatrix} 5 & 2 \\ 2 & 2 \end{pmatrix}$. $\lambda^2 - 7\lambda + 6 = 0 \Rightarrow \lambda = 6, 1$ (check: $6 + 1 = 7$, $6 \times 1 = 6$). $\lambda = 6$:

$-u_1 + 2u_2 = 0 \Rightarrow u = (2, 1)/\sqrt{5}$; $\lambda = 1$: $u = (-1, 2)/\sqrt{5}$. PC1 explains $6/7 = 85.7\%$. Scores $(2x_{c1} + x_{c2})/\sqrt{5} = (-8, -2, 0, 4, 6)/\sqrt{5}$, sample variance $120/20 = 6 = \lambda_1$.

Mistakes that cost marks

- **Not scaling first** — PCA then finds the column with the largest numbers.
- **Calling PCA feature selection.** Every component mixes every original feature; “Age was not useful” cannot be concluded from PCA.
- **Forgetting PCA is unsupervised:** it never looks at y , and the discarded low-variance direction may be the predictive one.
- **Unnormalised eigenvectors** in the scores.

Exercises 13 — numerical

- E13.1** Unit principal direction $u = (1/\sqrt{2}, 1/\sqrt{2})$; project the centred point $(2, 2)$ onto it.
- E13.2** Two standardised features have $r = 0.8$. Write the covariance matrix, both eigenvalues, and the variance explained by PC1.
- E13.3** Points $(1, 3), (5, 3), (5, 4), (7, 3), (7, 7)$: S (divisor $n - 1$), eigenvalues, unit eigenvectors, PC1’s share, and the PC1 scores.
- E13.4** Eigenvalues 5.2, 2.1, 1.1, 0.9, 0.4, 0.3. How many components keep 90% of the variance? 95%?

Short-answer questions, with model answers

- C13.1** *Does PCA keep a subset of the named columns?* No — it builds new columns, each a weighted mix of all of them.
- C13.2** *50 features: PCA keeps 5 components (92% variance); a colleague selects 5 original features. Which can you explain to a manager? Which helps k -NN more?* Selection — the features keep their meaning and units. PCA usually helps k -NN more: its components are uncorrelated and carry information from all 50 columns.
- C13.3** *Why might keeping 95% of the variance hurt prediction? How should k be chosen?* Variance in X is not relevance to y . Choose the number of components by cross-validated prediction error, inside the pipeline.
- C13.4** *Curse of dimensionality: what and whom?* As dimensions grow the space’s volume grows exponentially, data become sparse and all points nearly equidistant. k -NN, k -means and RBF SVMs suffer directly.
- C13.5** *3000 features, 200 rows?* Reduce (PCA or lasso) or regularise strongly, selecting inside cross-validation; a linear model could fit the training data perfectly.

14 Splitting and cross-validation

The method: honest evaluation

1. **Train / validation / test:** fit on training; choose hyperparameters on validation; report *once* on test.
2. **Split counts:** n rows at ratio $a:b \rightarrow na/(a+b)$ and $nb/(a+b)$.
3. **k -fold CV:** k folds of n/k rows; each is the validation fold once; k fits, each on $n(1 - 1/k)$ rows. Report the mean *and* the spread of the k scores. LOOCV is $k = n$.
4. **Match the split to the data:** *stratified* (keep class proportions in every fold), *grouped* (all rows of one patient or customer on one side), *time-ordered* (train on the

past, test on the future).

5. A random split of n rows with n_+ positives puts k positives in a test set of t rows with probability $\binom{n_+}{k} \binom{n-n_+}{t-k} / \binom{n}{t}$.

Example 14.1 — counts, and the mean and spread of CV

400 houses split 75:25: 300 training, 100 test. 5-fold CV on 100 rows: 20 validation rows per fold, 5 fits of 80 rows each.

Fold scores 0.78, 0.82, 0.75, 0.80, 0.85: mean $4.00/5 = 0.80$; deviations $-0.02, 0.02, -0.05, 0, 0.05$; squares sum to 0.0058; $\text{sd} = \sqrt{0.0058/5} = 0.0341$ (divisor k ; with $k-1$, 0.0381 — say which). Two models whose CV means differ by less than about this spread have not been shown to differ.

Example 14.2 — why stratify, group and respect time

Twenty patients, five ill, four held out. At random, the chance of *no* ill patient in the test set is $\binom{15}{4} / \binom{20}{4} = 1365/4845 = 0.2817$ — more than one split in four leaves recall uncomputable. Stratified: $5 \times \frac{4}{20} = 1$ ill and 3 healthy, every time.

Grouped. Repeated measurements of the same patient (or many transactions per customer) in both train and test let the model recognise individuals: the score is inflated. Split by patient (GroupKFold). **Time.** Stock prices split at random let the model train on the future and interpolate; use a temporal split.

Exercises 14 — numerical

- E14.1** 400 rows split 75:25: rows in the training portion?
E14.2 5-fold CV on 100 rows: validation rows per fold, and model fits?
E14.3 Fold scores 0.81, 0.79, 0.86, 0.74, 0.80: mean and sd (divisor k).
E14.4 30 rows, 6 positive, 5 held out at random: probability of no positive in the test set? What does a stratified split give?

Short-answer questions, with model answers

- C14.1** *Use of training, validation and test sets?* Training: learn the parameters. Validation: compare models and tune hyperparameters. Test: a single, final, unbiased estimate of performance on new data.
C14.2 *“My model is 96% accurate” — measured on training data. What is wrong, and what instead?* A training score measures fit, not generalisation, and rewards memorisation. Evaluate on a held-out test set (or by cross-validation).
C14.3 *Why a validation set when a test set exists?* Choosing a hyperparameter by the test score is fitting to the test set; after many comparisons the winner partly reflects that set’s noise. The validation set absorbs this so the test set stays clean.
C14.4 *Only 60 observations — how to estimate performance?* k -fold CV (or LOOCV); a single test set of a few rows is too unstable.
C14.5 *Tuning 50 models on the same CV scores and reporting the best?* Selection optimism: the best of 50 noisy scores is biased upward. Use nested CV, or confirm on an untouched test set.

15 Imbalanced data

The method: metrics and remedies for a rare class

1. Build the confusion matrix for the rare (positive) class: TP , FP , FN , TN .
2. accuracy = $\frac{TP+TN}{\text{all}}$; precision = $\frac{TP}{TP+FP}$ (of those flagged, how many were real); recall = $\frac{TP}{TP+FN}$ (of the real ones, how many were caught); $F_1 = \frac{2PR}{P+R}$.
3. Compare with the **always-negative** baseline: its accuracy equals the majority share, its recall is 0.
4. **Remedies**, applied to the training part only: random oversampling (duplicate minority rows), random undersampling (discard majority rows), SMOTE ($x_{\text{new}} = x_i + \lambda(x_{\text{nb}} - x_i)$, $\lambda \in [0, 1]$), class weights, a moved threshold.

Example 15.1 — one thousand patients, fifty ill

Always “no disease”: $TN = 950$, $FN = 50$, $TP = FP = 0$. Accuracy $950/1000 = 95\%$; recall $0/50 = 0$; precision $0/0$, undefined.

A real model flags 100 patients, 40 truly ill: $TP = 40$, $FP = 60$, $FN = 10$, $TN = 890$.

$$\text{precision} = \frac{40}{100} = 0.40, \quad \text{recall} = \frac{40}{50} = 0.80, \quad F_1 = \frac{2(0.40)(0.80)}{1.20} = 0.533,$$

$$\text{accuracy} = \frac{930}{1000} = 93\%.$$

Lower accuracy than the useless model, yet it catches 80% of the sick. In screening that trade is worth making; whether to favour precision or recall is a decision about consequences, not statistics.

Example 15.2 — resampling arithmetic, and SMOTE

From $950+50$: random oversampling $\rightarrow 950+950 = 1900$ rows (the 50 ill patients repeated); random undersampling $\rightarrow 50+50 = 100$ rows (900 patients, 90% of the data, thrown away). Balanced class weights $n/(Kn_c)$: $1000/1900 = 0.5263$ and $1000/100 = 10$.

SMOTE between $x_i = (5, 120)$ and its neighbour $(8, 150)$ with $\lambda = 0.25$: $x_i + 0.25(3, 30) = (5.75, 127.5)$. Every synthetic point lies on the segment between two real minority points — unsafe when the classes interleave, with categorical features, or in high dimensions.

Mistakes that cost marks

- **Reporting accuracy** on an imbalanced problem.
- **Resampling before the split**, or resampling the test set: copies or interpolations of test rows leak into training, and a resampled test set describes a world with the wrong prevalence.
- **Swapping precision and recall**.

Exercises 15 — numerical

- E15.1** Among 400 rows, 40 are positive: the positive percentage? The accuracy of an always-negative classifier?
- E15.2** Of 2000 transactions, 40 are fraudulent. A model flags 90 and catches 30 of the frauds. Confusion matrix, accuracy, precision, recall, F_1 .
- E15.3** 10,000 transactions, 100 fraudulent; a detector flags 150 and catches 60. Confusion matrix and all four metrics. A rival flags nothing: its accuracy? Which would you deploy?

- E15.4** 1200 majority and 80 minority rows: rows after random oversampling to balance? After undersampling? How many discarded?
- E15.5** 10,000 patients, 200 with a condition: accuracy and recall of “always healthy”?

Short-answer questions, with model answers

- C15.1** *400 houses, only 40 expensive: one sampling method; does it add or remove rows?*
Random oversampling (or SMOTE) adds minority rows; random undersampling removes majority rows.
- C15.2** *One application where recall matters more, one for precision.* Recall: disease screening (a miss is costly). Precision: automatically suspending accounts (a false alarm is costly).
- C15.3** *Oversampling vs SMOTE.* Oversampling duplicates existing rows (the model can memorise them); SMOTE interpolates new points (it asserts that the segment between two minority points is minority territory).

16 Data leakage and the order of the pipeline

The method: finding a leak

1. **Target leakage:** a feature that would not exist at the moment the prediction is made, or is computed from the target. Test: *at prediction time, would I have this value?*
2. **Preprocessing leakage:** any statistic (mean, median, min/max, quartiles, selected features, PCA axes, resampling) learned from rows that include the test set.
3. **The safe order:** understand the columns → drop leaks and identifiers → correct known errors → **split** → impute, transform, encode, scale, select, resample — all *fitted on training data* inside a **Pipeline** → model → evaluate once on test.
4. A pipeline prevents preprocessing leakage; only domain judgement prevents target leakage.

Example 16.1 — find every leak

```
df["price_per_sqft"] = df["Price_lakh"] / df["Area_sqft"]
df = df.fillna(df.mean(numeric_only=True))
X = StandardScaler().fit_transform(df.drop(columns=["Price_lakh"]))
X = SelectKBest(k=5).fit_transform(X, df["Price_lakh"])
```

- (i) `price_per_sqft` is computed from the target: **target leakage** — the model is handed the answer.
- (ii) `fillna(df.mean())` on all rows: **preprocessing leakage** (and the mean is a poor fill for skewed columns).
- (iii) `StandardScaler().fit_transform` on all rows: **preprocessing leakage**.
- (iv) `SelectKBest` with the labels of all rows: **preprocessing leakage** through feature selection — and it survives later cross-validation, because every fold inherits the same contaminated columns.

Also in the housing table: **Buyer_Income** is known only after the sale — a target leak; **ID** is an identifier — drop it (data reduction: feature selection).

Example 16.2 — the pipeline, in order

1. Drop ID and Income; correct H12's Age 150 $\rightarrow$ 15.
2. Split into training and test sets (stratify or group if needed).
3. On the training set only: median-impute Area and Age, group-wise mode for Garage; log-transform Price (or keep H11 and use a robust method); one-hot Location, ordinal Condition, binary Garage; engineer Area_per_Bedroom; scale if the model needs it.
4. Wrap step 3 and the model in a Pipeline, tune by cross-validation on the training set, then evaluate *once* on the test set.

Short-answer questions, with model answers

- C16.1** *scaler.fit_transform(X_test) appears in a pipeline.* It learns new parameters from the test data, so train and test are scaled differently and the test set influences the transformation. Use `fit_transform` on training, `transform` on test.
- C16.2** *A readmission model uses discharge_destination.* Target leakage for any model meant to predict at admission: the value is recorded at discharge and reflects the outcome being predicted.
- C16.3** *SMOTE on the whole dataset, then split, then 99% accuracy.* Synthetic points derived from rows that later land in the test set sit in training — the model has seen smeared copies of the answers.
- C16.4** *The 50 features most correlated with the target are selected, then the data are split and cross-validated; scores collapse in production.* Selection leakage: the test rows helped choose the features, and CV cannot catch it because selection happened before the loop. Put selection inside the pipeline.
- C16.5** *A house-price model uses the final registered sale tax.* Target leakage — the tax is computed from the sale price. Remove it and evaluate on data that mimics prediction time.
- C16.6** *Loan default (3% default): which column goes first — days_until_first_missed_payment or loan_officer_id?* The first: it exists only after a default has begun — a textbook target leak. The officer ID is an identifier and should also go.
- C16.7** *Why is leakage more dangerous than a crash?* It makes results look *better*, so every incentive is to accept it.
- C16.8** *Why must SMOTE, scaling and selection sit inside the CV loop?* So that every statistic is refitted on each fold's training part and the held-out fold stays unseen.

Unit V — Regression (L8–L9)

17 Regression foundations

The method: setting up a regression problem

1. **Regression** predicts a continuous numerical target from one or more features, by estimating f with $y \approx f(x)$ — for the data given and for new data.
2. **Two jobs:** *estimation* (how much does a square foot add?) and *prediction* (what will this 1750 sq ft house fetch?).
3. **“Linear” means linear in the parameters**, not in x : $b_0 + b_1x + b_2x^2$ and $b_0 + b_1 \ln x$ are linear models; $b_0e^{b_1x}$ is not.
4. **Eight steps:** state the question $\rightarrow$ collect and plot $\rightarrow$ choose the form $\rightarrow$ split $\rightarrow$

estimate (least squares) → check adequacy (residuals) → validate → interpret and report. If the residuals say the form is wrong, go back to step 3.

5. **Simple linear regression:** $y_i = \beta_0 + \beta_1 x_i + \varepsilon_i$; fitted $\hat{y}_i = b_0 + b_1 x_i$. Greek = unknown population values; Latin = estimates from the sample. ε_i (from the true line) is unobservable; the residual $e_i = y_i - \hat{y}_i$ is computed.
6. **Assumptions**, most to least damaging when violated: linearity, independence, homoscedasticity (constant $\text{Var } \varepsilon$), normality.

Example 17.1 — regression or not, linear or not

Model / task	Verdict
Predict tomorrow's closing price, yield per hectare, length of hospital stay	regression: the target is continuous
Predict whether a house is “expensive”	classification: the target is a category
$\hat{y} = b_0 + b_1 x + b_2 x^2$	linear regression: make a column x^2 and it is a two-feature linear model
$\hat{y} = b_0 + b_1 \ln x$	linear regression in the feature $\ln x$
$\hat{y} = b_0 e^{b_1 x}$	<i>not</i> linear: b_1 sits inside the exponential (taking $\ln y$ turns it into a linear model of $\ln y$)

β vs b , ε vs e . A different sample of 200 houses gives different b 's and the same β 's. The error term carries everything not in the model — condition of the plaster, the seller's urgency — and the model asks only that it be unsystematic.

Short-answer questions, with model answers

- C17.1** State the simple linear regression model and name every symbol. $y_i = \beta_0 + \beta_1 x_i + \varepsilon_i$: y target, x feature, β_0 intercept, β_1 slope, ε_i unobserved error, $i = 1, \dots, n$.
- C17.2** Why squared error to derive the estimates? Two reasons. It is differentiable and convex, giving a unique closed-form solution; and it punishes large misses more than small ones, which is usually wanted (and, by the Gauss–Markov theorem, gives the minimum-variance linear unbiased estimator). The price: one outlier can drag the line.
- C17.3** Name the four assumptions and rank them. Linearity, independence, homoscedasticity, normality — in that order of damage. Heteroscedasticity leaves the coefficients unbiased but the intervals wrong; normality matters only for small-sample inference.
- C17.4** Estimation vs prediction? Estimation asks for a trustworthy coefficient; prediction asks for a good $\hat{y}$ on new data. A model can do one well and the other badly.

18 Least squares by hand

The method: fitting $\hat{y} = b_0 + b_1 x$

1. Minimise $S(b_0, b_1) = \sum_i (y_i - b_0 - b_1 x_i)^2$.
2. **Normal equations** (set both partial derivatives to zero): $nb_0 + b_1 \sum x_i = \sum y_i$ and $b_0 \sum x_i + b_1 \sum x_i^2 = \sum x_i y_i$.
3. Solution: $b_1 = S_{xy}/S_{xx}$, $b_0 = \bar{y} - b_1 \bar{x}$, with $S_{xx} = \sum (x_i - \bar{x})^2 = \sum x_i^2 - n\bar{x}^2$ and $S_{xy} = \sum (x_i - \bar{x})(y_i - \bar{y}) = \sum x_i y_i - n\bar{x}\bar{y}$.
4. **Table:** $x - \bar{x}$, $y - \bar{y}$, $(x - \bar{x})^2$, $(x - \bar{x})(y - \bar{y})$, then $\hat{y}$, e , e^2 .

5. **Checks** (they are the two normal equations): $\sum e_i = 0$, $\sum x_i e_i = 0$; the line passes through $(\bar{x}, \bar{y})$; $SST = SSR + SSE$ with $SSR = b_1^2 S_{xx}$.

6. **Matrix form:** $b = (X^\top X)^{-1} X^\top y$ with a column of ones; $X^\top X = \begin{pmatrix} n & \sum x \\ \sum x & \sum x^2 \end{pmatrix}$,
 $X^\top y = \begin{pmatrix} \sum y \\ \sum xy \end{pmatrix}$.

Example 18.1 — derive b_0 and b_1 (Class Test Q4)

What we minimise: $S(b_0, b_1) = \sum_{i=1}^n (y_i - b_0 - b_1 x_i)^2$.

Differentiate.

$$\frac{\partial S}{\partial b_0} = -2 \sum (y_i - b_0 - b_1 x_i) = 0, \quad \frac{\partial S}{\partial b_1} = -2 \sum x_i (y_i - b_0 - b_1 x_i) = 0.$$

Normal equations. $nb_0 + b_1 \sum x_i = \sum y_i$ (1), and $b_0 \sum x_i + b_1 \sum x_i^2 = \sum x_i y_i$ (2).

Solve. Divide (1) by n : $b_0 + b_1 \bar{x} = \bar{y}$, so $b_0 = \bar{y} - b_1 \bar{x}$. Substitute into (2) and use $\sum x_i = n\bar{x}$:

$$b_1 (\sum x_i^2 - n\bar{x}^2) = \sum x_i y_i - n\bar{x}\bar{y} \implies b_1 = \frac{\sum x_i y_i - n\bar{x}\bar{y}}{\sum x_i^2 - n\bar{x}^2} = \frac{S_{xy}}{S_{xx}}.$$

Defined only when $S_{xx} > 0$ (the x values are not all equal). The second derivatives ($2n > 0$ and a positive determinant $4nS_{xx}$) confirm a minimum.

The line passes through $(\bar{x}, \bar{y})$: at $x = \bar{x}$, $\hat{y} = b_0 + b_1 \bar{x} = (\bar{y} - b_1 \bar{x}) + b_1 \bar{x} = \bar{y}$.

Example 18.2 — six houses by hand

house	A	B	C	D	E	F	sum
Area x	600	800	1000	1200	1400	1600	6600
Price y	40	61	74	81	87	107	450
$x - \bar{x}$	-500	-300	-100	100	300	500	0
$y - \bar{y}$	-35	-14	-1	6	12	32	0
$(x - \bar{x})^2$	250000	90000	10000	10000	90000	250000	700000
$(x - \bar{x})(y - \bar{y})$	17500	4200	100	600	3600	16000	42000
$\hat{y} = 9 + 0.06x$	45	57	69	81	93	105	450
$e = y - \hat{y}$	-5	+4	+5	0	-6	+2	0
e^2	25	16	25	0	36	4	106

$\bar{x} = 1100$, $\bar{y} = 75$; $b_1 = 42000/700000 = 0.06$ lakh per sq ft; $b_0 = 75 - 0.06 \times 1100 = 9$:

$$\hat{y} = 9 + 0.06x$$

Checks. $\sum e = 0$; $\sum xe = 0$; at $x = 1100$, $\hat{y} = 75$. $SSE = 106$, $MSE = 106/6 = 17.67$. $SST = S_{yy} = 2626$, $SSR = 2626 - 106 = 2520 = 0.06^2 \times 700000$ ✓. No other straight line has a smaller SSE: $S(b_0, b_1)$ is a convex bowl with one bottom at $(9, 0.06)$ — which is why calculus, not gradient descent, is enough here. (Gradient descent on the same SSE reaches the same answer, more slowly — and much faster when the features are scaled.)

Interpretation. Each extra square foot is *associated with* 0.06 lakh (Rs. 6,000) more; $b_0 = 9$ is the value at $x = 0$, far outside the data and not meaningful on its own.

Example 18.3 — the matrix route

$x = (0, 1, 2, 3)$, $y = (1, 3, 4, 6)$: $X^\top X = \begin{pmatrix} 4 & 6 \\ 6 & 14 \end{pmatrix}$, $X^\top y = \begin{pmatrix} 14 \\ 29 \end{pmatrix}$, $\det = 56 - 36 = 20$,

$$b = \frac{1}{20} \begin{pmatrix} 14 & -6 \\ -6 & 4 \end{pmatrix} \begin{pmatrix} 14 \\ 29 \end{pmatrix} = \frac{1}{20} \begin{pmatrix} 196 - 174 \\ -84 + 116 \end{pmatrix} = \begin{pmatrix} 1.1 \\ 1.6 \end{pmatrix}.$$

Cross-check: $\bar{x} = 1.5$, $\bar{y} = 3.5$, $S_{xx} = 5$, $S_{xy} = 8$, $b_1 = 1.6$, $b_0 = 1.1$ ✓.

How precise is b_1 ? $\text{Var}(b_1) = \sigma^2/S_{xx}$: spreading the x values out (larger S_{xx}) pins the slope down more precisely, at no extra cost in n .

Mistakes that cost marks

- $b_1 = S_{xy}/S_{yy}$ — that is the slope of x on y .
- **Rounding b_1 before computing b_0 .**
- **Residuals that do not sum to zero** mean an arithmetic error, every time (with an intercept).
- **Extrapolating** far outside the range of x , or reading b_1 causally.

Exercises 18 — numerical

- E18.1** $y = 12$, $\hat{y} = 10$: the squared residual. For six houses with $\text{SSE} = 106$: the MSE.
- E18.2** Least-squares slope and intercept for $(1, 2)$, $(2, 3)$, $(3, 5)$.
- E18.3** Use $\hat{y} = 9 + 0.06x$ at $x = 600$ with observed $y = 40$: the signed residual.
- E18.4** $x = (2, 4, 6, 8, 10)$, $y = (5, 11, 14, 20, 25)$: $\bar{x}$, $\bar{y}$, S_{xx} , S_{xy} , b_1 , b_0 ; predict at $x = 7$; verify the residuals sum to zero and compute SSE.
- E18.5** From summary statistics: $n = 10$, $\sum x = 50$, $\sum y = 120$, $\sum x^2 = 310$, $\sum xy = 660$. Find b_1 , b_0 and $\hat{y}(7)$.
- E18.6** Fit $\hat{y} = wx$ (no intercept) to $x = (1, 2, 3)$, $y = (4, 5, 7)$ with $w = \sum xy / \sum x^2$. Do the residuals sum to zero? Which property survives, and why?

Short-answer questions, with model answers

- C18.1** Write the two normal equations and say what each forces on the residuals. $\sum(y_i - b_0 - b_1x_i) = 0$ forces $\sum e_i = 0$; $\sum x_i(y_i - b_0 - b_1x_i) = 0$ forces $\sum x_i e_i = 0$ (residuals uncorrelated with x).
- C18.2** Why does changing area from sq ft to sq m change b_1 but not the correlation? $b_1 = r s_y/s_x$: r is unit-free, the slope carries the units (a factor of 10.76 here).
- C18.3** Model A: train RMSE 2, validation RMSE 9. Model B: 4 and 5. Which? B: A overfits — its training error flatters it.

19 Direct regression and maximum likelihood

The method: choosing “closest”, and the likelihood route to least squares

1. **Three defensible lines**, all through $(\bar{x}, \bar{y})$: *direct* (vertical distances, y on x) slope S_{xy}/S_{xx} ; *inverse* (x on y , drawn on y - x axes) slope S_{yy}/S_{xy} ; *orthogonal* (perpendicular distances) slope = direction of the leading eigenvector of the covariance matrix. Their ratio: direct/inverse = r^2 ; for $r > 0$ the orthogonal slope lies between the other two.
2. **Likelihood**. Assume $\varepsilon_i \sim N(0, \sigma^2)$ independent, so $y_i | x_i \sim N(\beta_0 + \beta_1 x_i, \sigma^2)$ with

density $f(y_i) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - \beta_0 - \beta_1 x_i)^2}{2\sigma^2}\right)$.

3. $L = \prod_i f(y_i)$; take logs: $\ln L = -\frac{n}{2} \ln(2\pi) - \frac{n}{2} \ln \sigma^2 - \frac{1}{2\sigma^2} \sum (y_i - \beta_0 - \beta_1 x_i)^2$.
4. **Punchline:** for any fixed σ^2 , maximising $\ln L$ over the coefficients = minimising SSE. Under normal errors, least squares *is* maximum likelihood.
5. $\partial \ln L / \partial \sigma^2 = 0$ gives $\hat{\sigma}_{\text{ML}}^2 = \text{SSE}/n$ — biased low. The unbiased estimate is $s^2 = \text{SSE}/(n - p - 1)$ (= $\text{SSE}/(n - 2)$ for one feature); s is the typical residual size, in the units of y .

Example 19.1 — three lines through the same five points

$x = (1, 2, 3, 4, 5)$, $y = (2, 4, 5, 4, 5)$: $\bar{x} = 3$, $\bar{y} = 4$, $S_{xx} = 10$, $S_{yy} = 6$, $S_{xy} = 6$.

Direct: $b_1 = 6/10 = 0.6$, $b_0 = 4 - 1.8 = 2.2$. **Inverse:** slope $S_{yy}/S_{xy} = 1.0$, intercept $4 - 3 = 1.0$. **Orthogonal:** covariance matrix $(n - 1 = 4) \begin{pmatrix} 2.5 & 1.5 \\ 1.5 & 1.5 \end{pmatrix}$, $\lambda = (4 \pm \sqrt{10})/2 = 3.5811, 0.4189$; leading eigenvector $(2.5 - 3.5811)v_1 + 1.5v_2 = 0 \Rightarrow \text{slope } v_2/v_1 = 0.7208$. Ratio $0.6/1.0 = 0.6 = r^2 = 36/60 \checkmark$, and $0.6 < 0.7208 < 1.0$. Each line minimises its own criterion and no other. “Regression” means the direct line; the next example shows why that criterion in particular. **Do not use the direct fit backwards:** rearranging $\hat{y} = 2.2 + 0.6x$ to predict x gives slope $1/0.6$, not the inverse regression.

Example 19.2 — from the likelihood to least squares, on six houses

In $\ln L$ the first two terms do not contain β_0, β_1 , and the third is $-\text{SSE}$ times the positive constant $1/(2\sigma^2)$. So the (β_0, β_1) that maximise $\ln L$ are exactly the least-squares (b_0, b_1) : for the six houses, $(9, 0.06)$.

Then $\frac{\partial \ln L}{\partial \sigma^2} = -\frac{n}{2\sigma^2} + \frac{\text{SSE}}{2\sigma^4} = 0 \Rightarrow \hat{\sigma}_{\text{ML}}^2 = \frac{\text{SSE}}{n} = \frac{106}{6} = 17.67$, while $s^2 = \frac{106}{6-2} = 26.5$, $s = 5.15$ lakh. The ratio $(n - 2)/n = 4/6$: at $n = 6$ the ML estimate is a third too small in expectation; at $n = 200$ the gap nearly vanishes.

Likelihood is a product of densities. Two independent observations with densities 0.2 and 0.3 under a candidate model have likelihood $0.2 \times 0.3 = 0.06$ ($\ln L = -2.8134$). A density height is not the probability of that exact value — for a continuous outcome that probability is 0.

Mistakes that cost marks

- **Dividing SSE by n and calling it unbiased.** That is the ML estimate; the unbiased one divides by $n - 2$ (one feature).
- **“Normality is needed for b_0, b_1 to be good.”** It is not: linearity, independence and constant variance suffice (Gauss–Markov). Normality gives the ML interpretation and exact small-sample tests.
- **Forgetting the logs:** maximise $\ln L$, not L — same maximiser, far easier algebra.
- **Unequal error variances:** if $\text{Var } \varepsilon_i$ differs by row, ML becomes *weighted* least squares.

Exercises 19 — numerical

E19.1 Independent observations have densities 0.2 and 0.3 under a candidate model. Their joint likelihood, and its log?

E19.2 For the line of Example 18.3 ($x = (0, 1, 2, 3)$, $y = (1, 3, 4, 6)$, $\hat{y} = 1.1 + 1.6x$) compute SSE, $\hat{\sigma}_{\text{ML}}^2$, s^2 and s .

- E19.3** For $x = (2, 4, 6, 8, 10)$, $y = (5, 11, 14, 20, 25)$ (E18.4) the direct slope is 2.45. Compute the inverse slope S_{yy}/S_{xy} and check that direct/inverse = r^2 .
- E19.4** Derive $\hat{\sigma}_{\text{ML}}^2 = \text{SSE}/n$ from $\ln L$.

20 R^2 , adjusted R^2 and the error summary

The method: judging how good a fit is

1. $\text{SST} = \sum (y_i - \bar{y})^2$ (the “predict $\bar{y}$ ” model), $\text{SSE} = \sum e_i^2$, $\text{SSR} = \sum (\hat{y}_i - \bar{y})^2 = b_1^2 S_{xx}$, and $\text{SST} = \text{SSR} + \text{SSE}$ (exact, because of the normal equations).
2. $R^2 = \text{SSR}/\text{SST} = 1 - \text{SSE}/\text{SST}$: the share of the variation in y the model accounts for. For one feature, $R^2 = r^2$ with $r = S_{xy}/\sqrt{S_{xx}S_{yy}}$ — use it as a check.
3. Adjusted: $R_{\text{adj}}^2 = 1 - (1 - R^2) \frac{n-1}{n-p-1}$, p = number of features (not counting the intercept). It can fall when a useless feature is added.
4. On held-out data use the test mean in SST; R^2 can then be *negative* (worse than predicting the mean).
5. **Report three numbers:** R^2 (share explained), $\text{RMSE} = \sqrt{\text{SSE}/n}$ (in the units of y , punishes big misses), MAE (typical miss).

Example 20.1 — the six houses

$\text{SST} = S_{yy} = 2626$, $\text{SSE} = 106$, $\text{SSR} = 2520 (= 0.06^2 \times 700000 \checkmark)$.

$$R^2 = \frac{2520}{2626} = 0.9596, \quad r = \frac{42000}{\sqrt{700000 \times 2626}} = 0.9796, \quad r^2 = 0.9596 \checkmark$$

$$R_{\text{adj}}^2 = 1 - (0.0404) \frac{5}{4} = 0.9495,$$

$$\text{RMSE} = \sqrt{106/6} = 4.20 \text{ lakh}, \quad \text{MAE} = \frac{22}{6} = 3.67 \text{ lakh}.$$

“Explains 96% of the variation” and “typically wrong by about 4 lakh” are both true; only the second tells a buyer whether the model is usable.

Example 20.2 — negative R^2 , and adjusted R^2 deciding

Held-out rows $y = (4, 6, 8, 10)$, predictions $(7, 6, 6, 5)$. Test mean 7: $\text{SST} = 9 + 1 + 1 + 9 = 20$; $\text{SSE} = 9 + 0 + 4 + 25 = 38$; $R^2 = 1 - 38/20 = -0.90$. Predicting 7 everywhere would score 0; R^2 has no lower bound. (Likewise $\text{SSE} = 150$, $\text{SST} = 100$ gives $R^2 = -0.5$.)

Two models, $n = 30$: A ($p = 2$, $R^2 = 0.64$) and B ($p = 8$, $R^2 = 0.70$). $R_{\text{adj}}^2(A) = 1 - 0.36 \times \frac{29}{27} = 0.6133$; $R_{\text{adj}}^2(B) = 1 - 0.30 \times \frac{29}{21} = 0.5857$ — A is preferred. At $n = 200$ the same R^2 s give 0.6363 and 0.6874 and B wins: the penalty weakens as n grows, so the final judge is still a held-out or cross-validated score.

Mistakes that cost marks

- “ $R^2 = 0.94$, so the model is 94% accurate.” R^2 is not accuracy (a classification term); it is unit-free and says nothing about the size of a typical error — and a training R^2 is not evidence at all.
- **Training R^2 rose when I added a feature, so it helps.** It can never fall: the old fit is still available with the new coefficient at 0.
- **Test R^2 with the training mean**, or p counting the intercept.

- R^2 as proof the form is right. Anscombe's four datasets share $R^2 \approx 0.67$; only one is a straight line with scatter.

Exercises 20 — numerical

- E20.1** SSE = 20 and SST = 100 on the same evaluation set: R^2 ? And for SSE = 150, SST = 100?
- E20.2** For the fit of E18.4 ($\hat{y} = 0.30 + 2.45x$): SSE, SST, R^2 (two ways), RMSE and MAE.
- E20.3** $n = 50$, $p = 6$, SST = 48,000, SSE = 7,200: R^2 , adjusted R^2 and $s = \sqrt{\text{SSE}/(n-p-1)}$. A seventh feature raises R^2 to 0.858: the new adjusted R^2 — did it earn its place?
- E20.4** Held-out $y = (2, 5, 8)$, predictions $(6, 5, 3)$: the test R^2 .
- E20.5** $n = 20$. A: $p = 1$, $R^2 = 0.50$. B: $p = 6$, $R^2 = 0.60$. Which does adjusted R^2 prefer?

21 Model adequacy, over-fitting and cross-validation

The method: checking a fitted regression

1. **Plot residuals against fitted values.** A formless band around 0 is healthy. A curve (smile or frown) $\Rightarrow$ wrong functional form (add x^2 , transform). A funnel $\Rightarrow$ non-constant variance (log the target, or weighted least squares). Isolated points $\Rightarrow$ investigate them; a high-leverage point (extreme in x) can rotate the line.
2. Residuals against each feature, against time (drift $\Rightarrow$ dependence), and a normal Q-Q plot (least important).
3. **Over-fitting:** great training score, much worse held-out score. **Under-fitting:** poor on both. Over-fitting depends on flexibility *relative to n* .
4. **Cross-validation as the detector:** choose the model (e.g. the polynomial degree) with the best cross-validated error; report the mean and spread across folds. Pool fold errors through MSE: pooled RMSE = $\sqrt{\text{mean of fold MSEs}}$ (equal-size folds).

Example 21.1 — the degree curve

Sixteen houses fitted, eight withheld; polynomials in area:

degree	train R^2	held-out R^2	verdict
1	0.811	+0.838	the relationship, correctly identified
3	0.820	+0.758	extra flexibility already costing accuracy
5	0.954	−37.5	fitting noise
9	0.979	−16,028	fitting noise energetically

Training R^2 climbs every time; the held-out score collapses. Every training-set score (R^2 , even adjusted R^2) prefers degree 9 — over-fitting is invisible from inside. The fix is procedural: withhold data and never let a decision depend on it. Choose **degree 1**.

Example 21.2 — reading fold scores

Five folds give RMSE 3.0, 2.5, 3.5, 4.0, 3.0 lakh. Mean fold RMSE = 3.2. Pooled RMSE = $\sqrt{(9 + 6.25 + 12.25 + 16 + 9)/5} = \sqrt{10.5} = 3.2404$ — not the same number, because RMSE is a square root of an average. The spread (2.5 to 4.0) is part of the answer: quoting one split's RMSE to four figures claims a precision the data does not have.

Model A: train RMSE 2, validation RMSE 9. **Model B:** 4 and 5. Choose **B**; A's training error flatters it (over-fitting).

Exercises 21 — numerical

- E21.1** Fold RMSEs are 2, 3, 4: their mean? Three equal folds have MSEs 4, 9, 16: the pooled out-of-fold RMSE?
- E21.2** Three models: A (train R^2 0.63, test 0.61), B (0.88, 0.85), C (0.999, 0.41). Diagnose each and choose.

Short-answer questions, with model answers

- C21.1** *A curved residual-vs-fitted pattern suggests?* The linear form is wrong (linearity violated); the coefficients describe a relationship that does not exist. Add a polynomial term or transform the feature.
- C21.2** *A funnel?* Non-constant variance: coefficients stay unbiased but their standard errors and intervals are wrong. Log the target or use weighted least squares.
- C21.3** *Distinguish nonlinearity, unequal variance and dependence in diagnostics. Can CV alone establish every assumption?* Curve vs funnel vs pattern over time/order. No — CV measures predictive error; it does not test independence or constant variance.
- C21.4** *Why is R^2 undefined when all evaluated targets are equal? Why can training R^2 rise with a useless predictor?* SST = 0, so the ratio divides by zero. Adding a column enlarges the space least squares searches, so SSE cannot rise.
- C21.5** *Adding a feature raises training R^2 from 0.81 to 0.83 but lowers CV R^2 from 0.79 to 0.74. Over-fitting: drop the feature; trust the cross-validated score.*
- C21.6** *Name Anscombe's point in one sentence.* Four datasets with the same means, line and R^2 look completely different when plotted — always plot.
- C21.7** *Leave-one-out or 5-fold?* 5- or 10-fold is the default; LOOCV (n fits) is nearly unbiased but high-variance, for very small datasets.

22 Practice papers

Sit each paper in 60 minutes without notes, then mark it against the solution. Both follow the class-test pattern: five questions, 20 marks, simple calculator allowed, all steps shown.

Class Test — Set B (as set on 1 September 2026)

- Q1. Standardisation** [3]
In a housing dataset the Price column has mean 80 lakh and standard deviation 20 lakh.
(a) Give one difference between normalisation (min-max) and standardisation (z -score).
(1) (b) Standardise three houses priced 100, 60 and 80 lakh. (2)
- Q2. Data reduction and sampling** [3]
A dataset has 400 houses and 15 columns, one of which is House_ID. (a) Why should House_ID be removed before training? Which preprocessing step is this? (2) (b) Only 40 of the 400 houses are marked expensive. Name one sampling method to fix this. Does it add rows or remove rows? (1)
- Q3. Model validation** [4]
(a) Give the use of the training, validation and test sets. (2) (b) A student says “my model is 96% accurate”, measured on the training data. Why is this number not correct, and what should he do instead? (1) (c) 400 houses are split 75:25. How many go in each part? (1)
- Q4. Finding the formulas for b_0 and b_1** [4]

We fit $\hat{y} = b_0 + b_1x$. Write down what is made as small as possible, derive b_0 and b_1 showing all steps, and show that the line passes through $(\bar{x}, \bar{y})$.

Q5. Fitting a straight line [6]

Area x (sq ft) 600 900 1200 1500 1800

Price y (lakh) 42 65 80 97 116

(a) Find $\bar{x}$, $\bar{y}$, S_{xx} and S_{xy} . (3) (b) Find b_1 and b_0 and write the line. (2) (c) Predict the price of a 1350 sq ft house. (1)

Set B — solutions

Q1. (a) Min-max maps to a fixed range $[0, 1]$ using the minimum and maximum; standardisation centres on the mean and divides by the standard deviation, giving mean 0, sd 1 and no fixed range (it is less distorted by a single extreme than min-max). (b) $z = (x - 80)/20$: $100 \rightarrow +1$; $60 \rightarrow -1$; $80 \rightarrow 0$.

Q2. (a) An identifier carries no information about price; a model can still latch onto it (memorising rows, or spurious order), which hurts generalisation. Removing it is *data reduction* — *feature selection* (dropping an irrelevant column). (b) Random oversampling (or SMOTE) of the expensive class — *adds* rows; or random undersampling of the rest — *removes* rows.

Q3. (a) Training: learn the parameters. Validation: compare models and tune hyperparameters. Test: one final, unbiased estimate on unseen data. (b) A training score measures how well the model fits data it has already seen and rewards memorisation (overfitting); evaluate on a held-out test set or by cross-validation. (c) 300 training, 100 test.

Q4. See Example 18.1: minimise $S = \sum(y_i - b_0 - b_1x_i)^2$; normal equations; $b_0 = \bar{y} - b_1\bar{x}$, $b_1 = S_{xy}/S_{xx}$; at $x = \bar{x}$, $\hat{y} = \bar{y}$.

Q5. (a) $\bar{x} = 6000/5 = 1200$, $\bar{y} = 400/5 = 80$. $x - \bar{x} = (-600, -300, 0, 300, 600)$; $y - \bar{y} = (-38, -15, 0, 17, 36)$. $S_{xx} = 360000 + 90000 + 0 + 90000 + 360000 = 900000$; $S_{xy} = 22800 + 4500 + 0 + 5100 + 21600 = 54000$. (b) $b_1 = 54000/900000 = 0.06$; $b_0 = 80 - 0.06 \times 1200 = 8$; $\hat{y} = 8 + 0.06x$. (Check: $\hat{y} = 44, 62, 80, 98, 116$; residuals $-2, 3, 0, -1, 0$, sum 0.) (c) $\hat{y}(1350) = 8 + 81 = 89$ lakh.

Practice Set C (new, same pattern)

Q1. Scaling [3]

Training Area has mean 1500 and sd 400 sq ft (population); its minimum is 1000 and maximum 3000. (a) Standardise areas 1900, 1100 and 1500. (1.5) (b) A test house has area 3400. Give its min-max scaled value, and say why the answer exceeding 1 is not an error. (1.5)

Q2. Imbalanced data [4]

600 loan applications, 60 of which defaulted. (a) What accuracy does “nobody defaults” achieve? What is its recall? (1) (b) A model gives $TP = 45$, $FN = 15$, $FP = 55$, $TN = 485$. Compute accuracy, precision, recall and F_1 . (2) (c) How many rows after random oversampling to balance? After random undersampling? Where in the workflow must resampling happen? (1)

Q3. Optimisers [4]

$J(w) = \frac{1}{2}(w - 5)^2$, $w_0 = 1$, $\eta = 0.5$. (a) Two steps of plain gradient descent. (1.5) (b) Two steps of momentum with $\beta = 0.5$ ($v \leftarrow \beta v + g$). (1.5) (c) The first Adam step with default β 's (ignore ϵ). (1)

Q4. Missing values and outliers [3]

A column reads 30, 34, ?, 38, 40, 41, 44, 46, 120. (a) Mean and median of the observed

values; which would you impute, and why? (1) (b) Quartiles (hinges), IQR and fences; which values are flagged? (1.5) (c) Name one way to keep the flagged value but reduce its influence. (0.5)

Q5. Fitting a straight line

[6]

Experience x (years)	1	3	5	7	9
Salary y (thousand/month)	29	37	42	49	53

(a) $\bar{x}$, $\bar{y}$, S_{xx} , S_{xy} . (2) (b) b_1 , b_0 , the line, and a check on the residuals. (1.5) (c) Predict the salary at 6 years. (1) (d) Compute SSE, R^2 and $s = \sqrt{\text{SSE}/(n-2)}$. (1.5)

Set C — solutions

Q1. (a) $z = (x-1500)/400$: 1900 $\rightarrow$ 1, 1100 $\rightarrow$ -1, 1500 $\rightarrow$ 0. (b) $(3400-1000)/2000 = 1.2$. The scaler is fitted on training data only, so a test value beyond the training maximum maps above 1; that correctly signals extrapolation. Refitting on the test set would be leakage.

Q2. (a) $540/600 = 90\%$; recall 0. (b) Accuracy $530/600 = 0.8833$; precision $45/100 = 0.45$; recall $45/60 = 0.75$; $F_1 = 2(0.45)(0.75)/1.20 = 0.5625$. Accuracy is *lower* than the useless model's, yet it finds 75% of defaulters. (c) Oversampling: $540+540 = 1080$; undersampling: $60+60 = 120$. Only on the training data, after the split (inside each CV fold).

Q3. $J'(w) = w - 5$. (a) $g = -4 \Rightarrow w = 1 + 2 = 3$; $g = -2 \Rightarrow w = 4$. (b) $v_1 = -4$, $w_1 = 3$; $g_2 = -2$, $v_2 = 0.5(-4) - 2 = -4$, $w_2 = 3 + 2 = 5$ — momentum reaches the minimum in two steps where GD is at 4. (c) $\hat{m}_1 = g_1 = -4$, $\hat{v}_1 = 16$, step $= -0.5(-4)/4 = +0.5$, $w_1 = 1.5$ — exactly η .

Q4. (a) Observed: 30, 34, 38, 40, 41, 44, 46, 120; mean $393/8 = 49.125$, median $(40+41)/2 = 40.5$. The 120 drags the mean above seven of eight values: impute the **median**. (b) Lower half 30, 34, 38, 40 $\Rightarrow Q_1 = 36$; upper half 41, 44, 46, 120 $\Rightarrow Q_3 = 45$; IQR = 9; fences 22.5 and 58.5: 120 is flagged. (c) Clip it to 58.5, log-transform the column, use a robust scaler, or use a robust loss/model.

Q5. (a) $\bar{x} = 5$, $\bar{y} = 42$; $x - \bar{x} = (-4, -2, 0, 2, 4)$, $y - \bar{y} = (-13, -5, 0, 7, 11)$; $S_{xx} = 40$, $S_{xy} = 52 + 10 + 0 + 14 + 44 = 120$. (b) $b_1 = 3$, $b_0 = 42 - 15 = 27$; $\hat{y} = 27 + 3x$; fitted 30, 36, 42, 48, 54, residuals -1, 1, 0, 1, -1, sum 0 $\checkmark$. (c) $\hat{y}(6) = 45$ thousand per month. (d) $\text{SSE} = 1+1+0+1+1 = 4$; $\text{SST} = S_{yy} = 169+25+0+49+121 = 364$; $R^2 = 1-4/364 = 0.9890$ (check: $r = 120/\sqrt{40 \times 364} = 0.9945$, $r^2 = 0.9890$); $s = \sqrt{4/3} = 1.1547$ thousand.

23 Answers to the numerical exercises

Rounded to four decimals at the end. Compare the *method* line by line, not only the final value. (The short-answer questions have their model answers beside them in the grey boxes.)

Exercises 1

E1.1 $5 + 0.06 \times 1000 = 65$ lakh. **E1.2** $120 \times 4 = 480$ entries; X is 120×4 , y has 120 entries. **E1.3** Distances 0.1414, 0.2236, 3.3838, 3.2311, 5.0090: nearest is (1.4, 0.2) — **setosa**. **E1.4** 94% are not readmitted, so predicting “not readmitted” for almost everyone scores about 94% while missing most real cases. Recall on the readmitted class (here 20%), or the confusion matrix, exposes it.

Exercises 2

E2.1 $50 - 47 = 3$. $40 - 45 = -5$: over-predicting by 5. **E2.2** $\text{MSE} = (1 + 4 + 16)/3 = 7$; $\text{MAE} = 7/3 = 2.3333$; Huber 0.5, 1.5, 3.5, mean $5.5/3 = 1.8333$. **E2.3** $|1| \leq 2$: 0.5; $|-3| > 2$: $2(3-1) = 4$; mean 2.25. **E2.4** $\text{MAE} = 3$, $\text{MSE} = (16 + 4)/2 = 10$. Squared penalty $9 \rightarrow 36$

($\times 4$); absolute $3 \rightarrow 6$ ($\times 2$). **E2.5** $e = (-1, 0, 2, -4)$: MSE = $21/4 = 5.25$, MAE = 1.75, RMSE = 2.2913; Huber 0.5, 0, 2, 6, mean 2.125. Row 4 supplies $16/21 = 76.2\%$ of the MSE. **E2.6** MSE: mean = 4; MAE: median = 2. With 112: mean 24 (moved by 20), median still 2.

Exercises 3

E3.1 $J'(5) = 2$; $w = 5 - 0.2 = 4.8$. **E3.2** $w_{\text{new}} - 3 = (1 - \eta)(w_{\text{old}} - 3)$: converges for $0 < \eta < 2$; one step at $\eta = 1$. **E3.3** $J'(1) = -1$: subtracting a negative gradient moves right, towards 2. From 5: $g = 3$, $w = 4.7$. **E3.4** Factor $1 - 2(0.2) = 0.6$. $w = 3, 1.4, 0.44, -0.136, -0.4816, -0.6890$; $f = 16, 5.76, 2.0736, 0.7465, 0.2687, 0.0967$. **E3.5** $J_0 = 11.6667$; gradient $(-6, -8.6667) \Rightarrow (b, w) = (0.6, 0.8667)$, $J = 3.2074$; gradient $(-3.0667, -4.5778) \Rightarrow (0.9067, 1.3244)$, $J = 0.8954$.

Exercises 4

E4.1 5. **E4.2** Row gradients 3 and 1; mean 2. **E4.3** Full gradient $(1 + (-1))/2 = 0$; SGD gradient for $y = 1$ is $+1$. Yes: with $\eta = 0.2$, $w \rightarrow 1.8$ and the full loss rises $0.5 \rightarrow 0.52$. **E4.4** $g = 3$, $w = 6 - 0.6 = 5.4$. **E4.5** $w^* = 26/14 = 1.8571$. Batch: mean of $(-16, -6, -30)$ is -17.3333 , $w = 0.8667$. SGD: 0.8, 1.02, 1.602 — much nearer. **E4.6** 200 updates per epoch, 2000 in all; 500,000 row-gradients.

Exercises 5

E5.1 $v_1 = 2$, $v_2 = 0.9(2) + 2 = 3.8$. **E5.2** $v_t = 2(1 + 0.9 + \dots + 0.9^{t-1}) = 20(1 - 0.9^t) \rightarrow 20$. **E5.3** $g = -6, v = -6, w = 0.6$; $g = -4.8, v = -7.8, w = 1.38$; $g = -3.24, v = -7.14, w = 2.094$ (GD: 1.464). **E5.4** 4, 6.2, 3.96, 1.168.

Exercises 6

E6.1 $9 + 16 = 25$; $w = 2 - 0.1 \times 4/5 = 1.92$. **E6.2** $V = 4$, $w = 5 - 0.1 \times 2/2 = 4.9$. From $V = 0$: $V = 0.4$, $\sqrt{V} = 0.6325$ (not $\sqrt{4}$). **E6.3** $g = 8, G = 64, w = 4.5$; $g = 7, G = 113, \sqrt{G} = 10.6301, w = 4.1707$; $g = 6.3415, G = 153.2146, \sqrt{G} = 12.3780, w = 3.9146$. **E6.4** Adagrad $G = 16, 20, 21, 21.25$; ratios 1, 0.4472, 0.2182, 0.1085. RMSprop $V = 1.6, 1.84, 1.756, 1.6054$; ratios 3.1623, 1.4744, 0.7546, 0.3946. RMSprop's stays larger: it forgets the early large gradients. **E6.5** $V = \rho^k V_0$. No: the numerator $g = 0$, so w does not move — only the history changes.

Exercises 7

E7.1 $m_1 = 0.2$, $v_1 = 0.004$, $\hat{m}_1 = 2$, $\hat{v}_1 = 4$, $w_1 = 4.9$. With $g_1 = -2$: $\hat{m}_1 = -2$, $\hat{v}_1 = 4$, step $+0.1$. **E7.2** $g = 4$: $m = 0.4$, $v = 0.016$, $\hat{m} = 4$, $\hat{v} = 16$, $w = 2.9$. $g = 3.8$: $m = 0.74$, $v = 0.030424$, $\hat{m} = 3.8947$, $\hat{v} = 15.2196$, $\sqrt{\hat{v}} = 3.9012$, $w = 2.8002$. **E7.3** $\hat{m}_1 = (1 - \beta_1)g_1/(1 - \beta_1) = g_1$, $\hat{v}_1 = g_1^2$, step $= -\eta g_1/|g_1|$. For $g_1 = -0.003$: $+0.01$. **E7.4** $(1 - \beta_1)/\sqrt{1 - \beta_2}$: $0.1/0.0316 = 3.1623\eta$; with $\beta_2 = 0.99$, $0.1/0.1 = 1\eta$.

Exercises 8

E8.1 $G = 4$, ratio $1/2$, $\Delta w = -1$. **E8.2** $G = 0.9 + 0.4 = 1.3$, $\sqrt{G} = 1.1402$; $\Delta w = -(0.5/1.1402)(-2) = +0.8771$; $U = 0.9(0.25) + 0.1(0.7692) = 0.3019$.

Exercises 9

E9.1 Median of 10, 20, 40 is 20. **E9.2** Mean $18000/6 = 3000$; median $(1800 + 2000)/2 = 1900$; impute the median — the 9000 makes the mean unrepresentative. **E9.3** Mean $299.5/11 = 27.23$, median 18; the 95 skews the mean, so fill with 18 (add a missing-indicator if missingness may be informative). H12 is a modest Prem Nagar house, so 18 is also plausible — though Income

is a leak and should be dropped anyway. **E9.4** Population $\text{sd} \times \sqrt{80/100} = 0.8944$; sample $\text{sd} \times \sqrt{79/99} = 0.8933$.

Exercises 10

E10.1 IQR = 8; upper fence $18 + 12 = 30$. **E10.2** $Q_1 = (25 + 30)/2 = 27.5$, $Q_3 = (45 + 50)/2 = 47.5$, IQR = 20, fences -2.5 and 77.5 : 200 flagged. **E10.3** Halves 12, 14, 15, 15 and 17, 18, 19, 60: $Q_1 = 14.5$, $Q_3 = 18.5$, IQR = 4, fences 8.5 and 24.5: flagged. $\bar{x} = 20.6667$, $s = 14.8997$, $z = 2.6399 < 3$: not flagged — and never could be, since the ceiling for $n = 9$ is $8/3 = 2.6667$; the 60 inflates the $\bar{x}$ and s it is judged by. Median 16, MAD = 2, $M = 0.6745 \times 44/2 = 14.84$: flagged. **E10.4** $9/\sqrt{10} = 2.8460$; $10/\sqrt{11} = 3.0151$. Only from $n = 11$ upwards. **E10.5** $\bar{x} = 2$, $m_2 = 16$, $m_3 = 96$, $g_1 = 96/64 = 1.5$ — exactly the ceiling $3/\sqrt{4}$: one point away from four equal ones is as skewed as five points can be.

Exercises 11

E11.1 $(100 - 80)/20 = 1$; $(40 - 10)/20 = 1.5$. **E11.2** $1500/4500 = 0.3333$. **E11.3** $\mu = 30$, $\sigma = \sqrt{1000/5} = 14.142$, $z_{50} = 1.4142$. **E11.4** $(60 - 37.5)/20 = 1.125$. **E11.5** Unscaled: $d_A = 50.04$, $d_B = 400$ — A. Scaled: Q (0.4, 0.5), A (0.425, 0), B (0.2, 0.5); $d_A = 0.5006$, $d_B = 0.2$ — B. **E11.6** Mean 4.6667, $\sigma = 6.3944$; standard $-1.3553, -0.7298, -0.4170, 0.2085, 0.5213, 1.7724$. Min-max $(v + 4)/20$: 0, 0.2, 0.3, 0.5, 0.6, 1. Median 4, $Q_1 = 0$, $Q_3 = 8$, robust $(v - 4)/8$: $-1, -0.5, -0.25, 0.25, 0.5, 1.5$. Max-abs $v/16$: $-0.25, 0, 0.125, 0.375, 0.5, 1$.

Exercises 12

E12.1 3. **E12.2** $6 - 1 + 5 = 10$; with drop-first 9. **E12.3** $\bar{y} = 20$, $S_{xy} = -10$, $S_{xx} = 2$: $\hat{y} = 25 - 5x$, fitted 25, 20, 15; SSE = $25 + 100 + 25 = 150$ (predicting the mean 20 everywhere gives 200). One-hot fits each mean exactly: SSE = 0.

Exercises 13

E13.1 $(2 + 2)/\sqrt{2} = 2.8284$. **E13.2** $\begin{pmatrix} 1 & 0.8 \\ 0.8 & 1 \end{pmatrix}$; $\lambda = 1.8, 0.2$; PC1 explains 90%. **E13.3** Mean (5, 4); $S = \begin{pmatrix} 6 & 2 \\ 2 & 3 \end{pmatrix}$; $\lambda^2 - 9\lambda + 14 = 0 \Rightarrow 7, 2$; $u_1 = (2, 1)/\sqrt{5}$, $u_2 = (-1, 2)/\sqrt{5}$; 77.8%; scores $(-9, -1, 0, 3, 7)/\sqrt{5} = (-4.0249, -0.4472, 0, 1.3416, 3.1305)$, variance 7. **E13.4** Cumulative 0.52, 0.73, 0.84, 0.93, 0.97: 4 for 90%, 5 for 95%.

Exercises 14

E14.1 300. **E14.2** 20 rows per fold; 5 fits. **E14.3** Mean 0.80; deviations 0.01, $-0.01, 0.06, -0.06, 0$; $\text{sd} = \sqrt{0.0074/5} = 0.0385$. **E14.4** $\binom{24}{5}/\binom{30}{5} = 42504/142506 = 0.2983$; stratified: 1 positive and 4 negatives.

Exercises 15

E15.1 10%; accuracy 90%. **E15.2** $TP = 30$, $FP = 60$, $FN = 10$, $TN = 1900$; accuracy 0.965, precision 0.3333, recall 0.75, $F_1 = 0.4615$. **E15.3** $TP = 60$, $FP = 90$, $FN = 40$, $TN = 9810$; accuracy 0.987, precision 0.40, recall 0.60, $F_1 = 0.48$. The rival: accuracy 99%, recall 0 — deploy the first. **E15.4** 2400 rows; 160 rows, discarding 1120. **E15.5** Accuracy 98%; recall 0.

Exercises 18

E18.1 4; $106/6 = 17.67$. **E18.2** $\bar{x} = 2$, $\bar{y} = 3.3333$, $S_{xx} = 2$, $S_{xy} = 3$; $b_1 = 1.5$, $b_0 = 0.3333$. **E18.3** $40 - 45 = -5$. **E18.4** $\bar{x} = 6$, $\bar{y} = 15$, $S_{xx} = 40$, $S_{xy} = 98$, $b_1 = 2.45$, $b_0 = 0.30$; $\hat{y}(7) = 17.45$. Residuals $-0.2, 0.9, -1.0, 0.1, 0.2$ (sum 0); $SSE = 1.90$. **E18.5** $S_{xx} = 310 - 250 = 60$, $S_{xy} = 660 - 600 = 60$, $b_1 = 1$, $b_0 = 12 - 5 = 7$, $\hat{y}(7) = 14$. **E18.6** $w = 35/14 = 2.5$; residuals $1.5, 0, -0.5$, sum $1 \neq 0$. $\sum xe = 0$ survives (the normal equation for w); “sum to zero” is the intercept’s normal equation, and there is no intercept.

Exercises 19

E19.1 $0.2 \times 0.3 = 0.06$; $\ln 0.06 = -2.8134$. **E19.2** Residuals $-0.1, 0.3, -0.3, 0.1$; $SSE = 0.2$; $\hat{\sigma}_{ML}^2 = 0.05$; $s^2 = 0.2/2 = 0.1$; $s = 0.3162$. **E19.3** $S_{yy} = 242$, $S_{xy} = 98$: inverse slope $242/98 = 2.4694$; $2.45/2.4694 = 0.9921 = 98^2/(40 \times 242) = r^2$ ✓. **E19.4** $\partial \ln L / \partial \sigma^2 = -\frac{n}{2\sigma^2} + \frac{SSE}{2\sigma^4} = 0 \Rightarrow \sigma^2 = SSE/n$.

Exercises 20

E20.1 $1 - 20/100 = 0.8$; $1 - 150/100 = -0.5$ (worse than predicting the mean). **E20.2** $SSE = 1.90$; $SST = 100 + 16 + 1 + 25 + 100 = 242$; $R^2 = 1 - 1.90/242 = 0.9921$ ($r = 98/\sqrt{9680} = 0.9961$, $r^2 = 0.9921$); $RMSE = \sqrt{1.90/5} = 0.6164$; $MAE = 2.40/5 = 0.48$. **E20.3** $R^2 = 0.85$; $R_{adj}^2 = 1 - 0.15 \times 49/43 = 0.8291$; $s = \sqrt{7200/43} = 12.94$. With $p = 7$: $1 - 0.142 \times 49/42 = 0.8343$ — it rose, so on this criterion the feature earned its place (confirm on held-out data). **E20.4** Test mean 5: $SST = 18$, $SSE = 16 + 0 + 25 = 41$, $R^2 = -1.2778$. **E20.5** A: $1 - 0.5 \times 19/18 = 0.4722$; B: $1 - 0.4 \times 19/13 = 0.4154$. A.

Exercises 21

E21.1 Mean 3; pooled $\sqrt{(4 + 9 + 16)/3} = \sqrt{29/3} = 3.1091$. **E21.2** A under-fits (both low, small gap); B fits well; C over-fits (near-perfect training, collapsed test). Deploy B; for A add features or flexibility.

The night before

- Redo one worked example per section *without looking*: a Huber table, a three-step Adam table, the IQR fences, a KNN distance before and after scaling, a 2×2 PCA, a confusion matrix, the six-house fit.
- Rehearse the derivation of b_0 and b_1 , and the step from the Gaussian log-likelihood to least squares, until you can write them unprompted — both are favourite 4-mark questions.
- For any fitted line, be able to produce SSE , SST , SSR , R^2 , adjusted R^2 , $\hat{\sigma}_{ML}^2$, s , $RMSE$ and MAE in one pass.
- Learn the one-line distinctions: selection cuts columns, sampling cuts rows; normalise to a range, standardise to a mean; an error is wrong, an outlier is rare; validation chooses, test reports once; resample the training set only.
- Out of syllabus for this paper: cross-entropy, hinge and triplet losses, and everything from Lecture 10 on (logistic regression, multiple regression, regularisation).